

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN 1

TÊN ĐỀ TÀI:
HỆ THỐNG GIÁM SÁT VÀ HỖ TRỢ BẢO TRÌ MÁY CÔNG
NGHIỆP ỨNG DỤNG IIoT
NGÀNH: KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

HỘI ĐỒNG: HD40

GVHD: PGS. TS. TRƯƠNG QUANG VINH

SVTH: THÁI ĐỨC THIÊN - 2313222

TP. HỒ CHÍ MINH, THÁNG 6 NĂM 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn giảng viên hướng dẫn đã định hướng, góp ý và hỗ trợ em trong quá trình thực hiện đồ án. Em cũng cảm ơn các thầy cô trong khoa/bộ môn đã trang bị nền tảng kiến thức về hệ thống nhúng, cảm biến, truyền thông và xử lý tín hiệu, giúp em có cơ sở để triển khai sản phẩm thực tế. Trong quá trình thực hiện, em đã học được cách phân tích yêu cầu, lựa chọn linh kiện, thiết kế phần cứng, lập trình firmware, xây dựng phần mềm giám sát và kiểm thử hệ thống. Do giới hạn về thời gian và điều kiện thiết bị, đồ án khó tránh khỏi thiếu sót; em mong nhận được góp ý để hoàn thiện sản phẩm trong các phiên bản tiếp theo.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án trình bày quá trình thiết kế và chế tạo một hệ thống giám sát hỗ trợ bảo trì máy công nghiệp theo hướng IIoT. Sản phẩm thực tế là một nút cảm biến không dây sử dụng ESP32-WROOM-32U, MPU6050 và AHT30 để thu thập rung động, nhiệt độ, độ ẩm và các đặc trưng vận hành liên quan. Dữ liệu được xử lý sơ bộ tại biên, trong đó có tính toán RMS và phân tích phổ FFT nhằm trích xuất đặc trưng tần số của rung động. Sau đó, dữ liệu được đóng gói dạng JSON và truyền qua Wi-Fi/TCP Socket về phần mềm giám sát trung tâm được xây dựng bằng C# WinForms kết hợp DevExpress Chart/Dashboard. Báo cáo cũng trình bày cấu trúc kiểm thử sản phẩm, bảng linh kiện, quy trình đánh giá chức năng, các hình ảnh minh chứng thực nghiệm và hướng phát triển hệ thống. Kết quả đồ án cho thấy phương án có tính khả thi ở quy mô phòng thí nghiệm, phù hợp làm nền tảng cho hệ thống giám sát tình trạng thiết bị và bảo trì dự đoán chi phí thấp.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN	1
TÓM TẮT NỘI DUNG ĐỒ ÁN	1
MỤC LỤC	2
DANH MỤC BẢNG BIỂU	5
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	6
1.1. Đặt vấn đề.....	6
1.2. Tình hình nghiên cứu trong và ngoài nước.....	6
1.3. Mục tiêu đề tài	7
1.4. Phạm vi thực hiện	7
1.5. Phương pháp thực hiện	7
1.6. Hướng triển khai của đề tài.....	8
1.7. Điểm khác biệt của hệ thống.....	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	10
2.1. Bảo trì dự đoán và giám sát tình trạng máy	10
2.2. Rung động cơ khí và phân tích phổ FFT.....	10
2.3. Edge Computing trên ESP32.....	11
2.4. Truyền thông Wi-Fi/TCP Socket và định dạng JSON.....	11
2.5. Giao diện giám sát C# WinForms và DevExpress.....	12
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG.....	14
3.1. Kiến trúc tổng quát của sản phẩm thực tế.....	14
3.2. Lựa chọn phần cứng và danh sách linh kiện	15
3.3. Tính toán thời lượng sử dụng pin	19
3.3.1. Thông số pin và giả thiết tính toán	19
3.3.2. Thời gian hoạt động trong một ngày	20
3.3.3. Tính dòng tiêu thụ trung bình	20
3.3.4. Ước lượng thời lượng pin	21
3.4. Thiết kế cơ khí và bố trí vỏ hộp.....	22
3.5. Thiết kế phần mềm nhúng.....	22
3.6. Cấu trúc gói dữ liệu JSON.....	26
3.7. Thiết kế phần mềm giám sát trung tâm	27
3.8. Ưu điểm và hạn chế của phương án thiết kế.....	29
3.8.1. Ưu điểm của phương án thiết kế	29
3.8.2. Hạn chế của phương án thiết kế.....	30

CHƯƠNG 4. THI CÔNG SẢN PHẨM VÀ QUY TRÌNH KIỂM THỬ	31
4.1. Quy trình thi công sản phẩm	31
4.2. Kế hoạch kiểm thử chức năng	32
4.3. Tiêu chí đánh giá sản phẩm	33
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	35
5.1. Kết quả triển khai sản phẩm	35
5.2. Kết quả kiểm thử dữ liệu cảm biến	36
5.3. Kết quả phân tích FFT và giả lập lỗi	38
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
6.1. Kết luận	41
6.2. Hướng phát triển	41
TÀI LIỆU THAM KHẢO	43
PHỤ LỤC	44

DANH MỤC HÌNH VẼ

Hình 3.1. Sơ đồ khối kiến trúc phần cứng của hệ thống	14
Hình 3.2. Hình ảnh tổng thể sản phẩm thực tế sau khi lắp ráp hoàn chỉnh.....	15
Hình 3.3. Board ESP32 sử dụng trong nút cảm biến	16
Hình 3.4. Cảm biến rung động MPU6050.....	16
Hình 3.5. Cảm biến nhiệt độ và độ ẩm AHT30.....	17
Hình 3.6. Khối nguồn sử dụng cho nút cảm biến.....	17
Hình 3.7. Mạch giảm áp MP1584EN 3A sử dụng trong khối nguồn của nút cảm biến.....	18
Hình 3.8. Lưu đồ xử lý firmware ESP32.....	23
Hình 3.9. Lưu đồ xử lý cảm biến.....	24
Hình 3.10. Lưu đồ truyền thông Wifi TCP Socket.....	25
Hình 3.11. Đoạn chương trình tạo gói JSON dữ liệu rung động.....	26
Hình 3.12. Đoạn chương trình tạo gói JSON trạng thái hệ thống	27
Hình 3.13. Giao diện phần mềm giám sát trung tâm khi nhận dữ liệu từ ESP32	29
Hình 4.1. Quá trình thi công và kiểm tra mạch của nút cảm biến.....	31
Hình 5.1. Sản phẩm nút cảm biến sau khi hoàn thiện	35
Hình 5.2. Sản phẩm hoạt động và gửi dữ liệu trong quá trình thử nghiệm.....	36
Hình 5.3. Kết quả đo nhiệt độ và độ ẩm từ cảm biến AHT30.....	37
Hình 5.4. Kết quả đo rung động từ cảm biến MPU6050.....	38
Hình 5.5. Phổ FFT ở trạng thái hoạt động bình thường	38
Hình 5.6. Kết quả FFT khi giả lập mất cân bằng cơ khí	39
Hình 5.7. Kết quả FFT khi giả lập rung nhanh hoặc ma sát.....	39

DANH MỤC BẢNG BIỂU

Bảng 3.1. Danh sách linh kiện sử dụng trong hệ thống.....	18
Bảng 3.2. Thông số cụm pin sử dụng trong tính toán	19
Bảng 3.3. Giả thiết chu kỳ hoạt động và Deep Sleep của thiết bị	20
Bảng 3.4. Thông số dòng điện sử dụng để tính I_{avg}	21
Bảng 3.5. Các thành phần giao diện giám sát đề xuất.....	28
Bảng 4.1. Kế hoạch kiểm thử sản phẩm thực tế	32
Bảng 5.1. Bảng ghi nhận kết quả kiểm thử cảm biến.....	36

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Trong bối cảnh chuyển đổi số và phát triển nhà máy thông minh, việc theo dõi tình trạng máy móc theo thời gian thực đóng vai trò quan trọng trong duy trì năng suất, giảm thời gian dừng máy và hạn chế chi phí bảo trì. Với phương pháp bảo trì truyền thống, thiết bị thường chỉ được sửa chữa sau khi hỏng hoặc được kiểm tra theo lịch cố định. Cách tiếp cận này chưa phản ánh đầy đủ trạng thái vận hành thực tế, đặc biệt đối với các lỗi xuất hiện dần như mất cân bằng, lệch trục, mòn ổ bi, lỏng cơ khí hoặc quá nhiệt.

Các dấu hiệu bất thường của máy công nghiệp thường biểu hiện thông qua sự thay đổi rung động, nhiệt độ, độ ẩm môi trường và dòng điện/tải tiêu thụ. Vì vậy, việc xây dựng một nút cảm biến nhỏ gọn có khả năng gắn trực tiếp lên vỏ máy, thu thập dữ liệu liên tục hoặc theo chu kỳ, xử lý tại biên và gửi dữ liệu về máy tính giám sát là hướng triển khai phù hợp với quy mô đồ án môn học.

1.2. Tình hình nghiên cứu trong và ngoài nước

Trong những năm gần đây, bảo trì dự đoán và giám sát tình trạng máy đã trở thành một hướng nghiên cứu quan trọng trong lĩnh vực công nghiệp thông minh. Trên thế giới, nhiều hệ thống giám sát thiết bị quay như động cơ, quạt, bơm, hộp số và máy nén đã sử dụng cảm biến rung động, cảm biến nhiệt độ, cảm biến dòng điện kết hợp với các thuật toán xử lý tín hiệu để phát hiện sớm dấu hiệu bất thường. Trong đó, phân tích rung động là một phương pháp phổ biến vì các lỗi cơ khí như mất cân bằng, lệch trục, lỏng cơ khí hoặc hư hỏng ổ bi thường làm thay đổi biên độ và thành phần tần số của tín hiệu rung. Các nghiên cứu tổng quan về giám sát rung động cho thấy quy trình thường gồm các bước: thu thập dữ liệu, truyền dữ liệu, xử lý tín hiệu và phát hiện lỗi.

Bên cạnh các hệ thống đo chuyên dụng trong công nghiệp, xu hướng hiện nay là kết hợp IoT, Edge Computing và trí tuệ nhân tạo để tạo ra các hệ thống giám sát linh hoạt hơn. Dữ liệu từ cảm biến có thể được xử lý một phần tại thiết bị biên nhằm giảm dung lượng truyền, giảm độ trễ và tăng khả năng phản hồi cục bộ. Một số nghiên cứu gần đây đã đề xuất hệ thống giám sát tình trạng máy dựa trên Edge Computing, trong đó dữ liệu từ lớp cảm biến được xử lý ngay tại thiết bị hoặc gần thiết bị trước khi gửi về hệ thống trung tâm.

Ở hướng chi phí thấp, nhiều mô hình thử nghiệm sử dụng vi điều khiển tích hợp Wi-Fi như ESP32 kết hợp với cảm biến MEMS để đo rung động, âm thanh hoặc thông số môi trường. Các hệ thống này phù hợp với máy quay công suất nhỏ, mô hình phòng thí nghiệm hoặc các bài toán cần triển khai nhanh với chi phí hạn chế. Một số công trình đã chứng minh khả năng sử dụng ESP32 và cảm biến MEMS để thu thập rung động, thực hiện các bước xử lý như RMS, FFT và gửi dữ liệu về hệ thống giám sát phục vụ bảo trì dự đoán.

Tại Việt Nam, bảo trì dự đoán cũng đang được quan tâm trong bối cảnh các doanh nghiệp từng bước chuyển đổi số và ứng dụng IIoT vào sản xuất. Một số tài liệu và bài viết trong nước nhấn mạnh vai trò của cảm biến IoT, phân tích dữ liệu, giám sát rung động, nhiệt độ và các thông số vận hành để phát hiện sớm tình trạng bất thường của máy móc. Ngoài ra, đã có nghiên cứu trong nước đề cập đến việc ứng dụng AI và IoT để

giám sát, chẩn đoán lỗi động cơ điện, trong đó tín hiệu rung động và nhiệt độ được xem là các nguồn dữ liệu quan trọng.

Tuy nhiên, nhiều hệ thống giám sát công nghiệp hiện nay vẫn có một số hạn chế khi áp dụng cho quy mô nhỏ hoặc mục đích đào tạo. Các thiết bị chuyên dụng thường có chi phí cao, yêu cầu cảm biến công nghiệp chính xác, hạ tầng truyền thông ổn định và phần mềm phân tích phức tạp. Trong khi đó, các mô hình học thuật hoặc thử nghiệm thường chỉ tập trung vào một phần của hệ thống, chẳng hạn chỉ đo rung động, chỉ hiển thị dữ liệu hoặc chỉ mô phỏng thuật toán, chưa thể hiện đầy đủ chuỗi xử lý từ cảm biến đến giao diện giám sát.

Từ tình hình trên, đề tài lựa chọn hướng xây dựng một hệ thống giám sát ở quy mô thực nghiệm, có chi phí hợp lý và dễ triển khai. Sản phẩm tập trung vào nút cảm biến không dây sử dụng ESP32, cảm biến rung động MPU6050, cảm biến nhiệt độ - độ ẩm AHT30, xử lý dữ liệu tại biên và truyền dữ liệu về phần mềm giám sát C# WinForms. Hướng triển khai này không nhằm thay thế các thiết bị chẩn đoán công nghiệp chuyên dụng, mà đóng vai trò mô hình minh chứng nguyên lý cho bài toán giám sát tình trạng máy và hỗ trợ bảo trì dự đoán trong môi trường phòng thí nghiệm hoặc xưởng nhỏ.

1.3. Mục tiêu đề tài

Một là, thiết kế và chế tạo một nút cảm biến không dây phục vụ giám sát trạng thái máy móc công nghiệp.

Hai là, thu thập dữ liệu rung động 3 trục, nhiệt độ và độ ẩm môi trường bằng các cảm biến tích hợp.

Ba là, thực hiện xử lý tín hiệu tại biên, bao gồm lọc nhiễu, tính RMS và phân tích phổ FFT để trích xuất đặc trưng rung động.

Bốn là, truyền dữ liệu dạng JSON qua Wi-Fi/TCP Socket về phần mềm giám sát trung tâm.

Năm là, xây dựng giao diện C# WinForms/DevExpress để hiển thị dữ liệu thời gian thực, biểu đồ môi trường, biểu đồ rung động và phổ tần số.

Sáu là, đề xuất quy trình kiểm thử sản phẩm thực tế, bao gồm minh chứng ảnh chụp, bảng linh kiện, bảng kết quả và đánh giá khả năng phát triển.

1.4. Phạm vi thực hiện

Đề án tập trung vào mô hình thử nghiệm quy mô nhỏ, phù hợp kiểm chứng nguyên lý bảo trì dự đoán trong phòng lab. Sản phẩm hiện tại ưu tiên giám sát rung động và môi trường bằng nút cảm biến không dây. Các chức năng phân tích chuyên sâu bằng học máy, triển khai nhiều nút cảm biến đồng thời, lưu trữ cơ sở dữ liệu dài hạn hoặc tích hợp chuẩn công nghiệp đầy đủ được xem là hướng phát triển tiếp theo.

1.5. Phương pháp thực hiện

Phân tích yêu cầu bài toán bảo trì máy công nghiệp và lựa chọn thông số giám sát phù hợp.

Thiết kế kiến trúc phần cứng gồm khối xử lý, khối cảm biến, khối nguồn, khối truyền thông và vỏ hộp cơ khí.

Lập trình firmware ESP32 để lấy mẫu dữ liệu, tính toán đặc trưng, đóng gói JSON và truyền dữ liệu không dây.

Xây dựng phần mềm giám sát trung tâm bằng C# WinForms và DevExpress để trực quan hóa dữ liệu.

Thi công sản phẩm, kiểm thử từng khối chức năng và đánh giá hệ thống bằng các kịch bản giả lập lỗi an toàn.

1.6. Hướng triển khai của đề tài

Đề tài được triển khai theo hướng xây dựng một hệ thống giám sát tình trạng máy móc ở quy mô thực nghiệm, trong đó trọng tâm là thiết kế một nút cảm biến không dây có khả năng thu thập dữ liệu, xử lý sơ bộ và truyền thông tin về phần mềm giám sát trung tâm. Cách tiếp cận này phù hợp với mục tiêu tạo ra một sản phẩm có thể hoạt động thực tế, dễ lắp đặt và có khả năng mở rộng trong các phiên bản sau.

Về phần cứng, hệ thống sử dụng ESP32 làm bộ xử lý trung tâm nhờ khả năng tích hợp Wi-Fi, kích thước nhỏ gọn và phù hợp với các ứng dụng IoT. Cảm biến MPU6050 được dùng để thu thập dữ liệu rung động theo ba trục, phục vụ việc đánh giá dao động cơ khí của thiết bị. Cảm biến AHT30 được sử dụng để đo nhiệt độ và độ ẩm môi trường xung quanh cụm máy. Ngoài ra, thiết bị còn có khối nguồn pin, anten ngoài và vỏ hộp bảo vệ nhằm hỗ trợ quá trình thử nghiệm trong điều kiện thực tế.

Về phần mềm nhúng, ESP32 thực hiện các nhiệm vụ chính gồm khởi tạo cảm biến, lấy mẫu dữ liệu, tính toán giá trị đặc trưng, xác định trạng thái cảnh báo, đóng gói dữ liệu theo định dạng JSON và truyền về máy tính giám sát. Một số bước xử lý được thực hiện ngay tại thiết bị nhằm giảm lượng dữ liệu truyền đi và giúp hệ thống phản hồi nhanh hơn khi xuất hiện dấu hiệu bất thường.

Về phần mềm giám sát, đề tài sử dụng C# WinForms kết hợp DevExpress để xây dựng giao diện trực quan trên máy tính. Phần mềm có nhiệm vụ nhận dữ liệu từ ESP32, phân tách gói JSON và hiển thị các thông số như nhiệt độ, độ ẩm, rung động, trạng thái pin, trạng thái cảnh báo và các thông tin liên quan đến quá trình hoạt động của nút cảm biến.

Quá trình triển khai được thực hiện theo các bước chính: lựa chọn linh kiện, thiết kế kiến trúc hệ thống, lắp ráp phần cứng, lập trình firmware ESP32, xây dựng phần mềm giám sát, kiểm thử từng khối chức năng và đánh giá kết quả trên mô hình thử nghiệm. Hướng triển khai này giúp đồ án thể hiện đầy đủ chuỗi xử lý của một hệ thống IIoT cơ bản, từ cảm biến đến phần mềm giám sát, đồng thời tạo nền tảng cho các hướng phát triển như lưu trữ dữ liệu dài hạn, mở rộng nhiều nút cảm biến hoặc ứng dụng thuật toán học máy cho bảo trì dự đoán.

1.7. Điểm khác biệt của hệ thống

So với các hệ thống giám sát công nghiệp chuyên dụng, đề tài được triển khai theo hướng đơn giản hơn, chi phí thấp hơn và phù hợp với quy mô thực nghiệm. Thay vì sử dụng các thiết bị đo rung động công nghiệp có giá thành cao hoặc hệ thống máy chủ phức tạp, sản phẩm tập trung vào việc xây dựng một nút cảm biến không dây nhỏ gọn, có thể gắn trực tiếp lên máy để thu thập dữ liệu và truyền về phần mềm giám sát trên máy tính.

Điểm khác biệt đầu tiên của hệ thống là khả năng lắp đặt linh hoạt. Nút cảm biến được thiết kế theo dạng thiết bị độc lập, có vỏ bảo vệ, nguồn pin và cơ cấu giá bằng nam châm. Nhờ đó, thiết bị có thể được đặt thử nghiệm ở nhiều vị trí khác nhau trên thân

máy, quạt hoặc động cơ mà không cần khoan, bắt vít hoặc can thiệp vào kết cấu cơ khí của thiết bị cần giám sát.

Điểm khác biệt thứ hai là hệ thống có xử lý dữ liệu tại thiết bị biên. ESP32 không chỉ đọc dữ liệu cảm biến mà còn thực hiện các bước xử lý sơ bộ như tính độ lớn rung động, xác định trạng thái cảnh báo và trích xuất thông tin đặc trưng trước khi truyền dữ liệu. Cách làm này giúp giảm lượng dữ liệu gửi về máy tính, đồng thời giúp hệ thống phản hồi nhanh hơn khi xuất hiện dấu hiệu bất thường.

Bên cạnh đó, phần mềm giám sát được xây dựng bằng C# WinForms kết hợp DevExpress, phù hợp với mô hình chạy trực tiếp trên máy tính cá nhân. Người dùng có thể quan sát các thông số như nhiệt độ, độ ẩm, rung động, trạng thái pin và cảnh báo thông qua giao diện trực quan. Cách triển khai này giúp giảm độ phức tạp so với mô hình Web Server hoặc Cloud, đồng thời thuận tiện cho quá trình thử nghiệm và trình diễn sản phẩm trong phạm vi đề án.

Ngoài ra, hệ thống có tính mở rộng tương đối tốt. Cấu trúc dữ liệu JSON cho phép bổ sung thêm trường dữ liệu mới khi cần tích hợp thêm cảm biến. Phần truyền thông có thể phát triển từ TCP Socket sang MQTT để phù hợp hơn với mô hình nhiều nút cảm biến trong tương lai. Phần mềm giám sát cũng có thể mở rộng thêm chức năng lưu lịch sử, xuất báo cáo, thống kê lỗi hoặc kết nối cơ sở dữ liệu.

Tóm lại, điểm nổi bật của đề tài không nằm ở việc thay thế hoàn toàn các hệ thống chẩn đoán công nghiệp chuyên dụng, mà ở việc xây dựng được một mô hình giám sát có chi phí hợp lý, dễ triển khai, có xử lý dữ liệu tại biên và có khả năng trình diễn rõ nguyên lý của bảo trì dự đoán. Đây là hướng tiếp cận phù hợp với môi trường học tập, phòng thí nghiệm và các bài toán giám sát máy móc quy mô nhỏ.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Bảo trì dự đoán và giám sát tình trạng máy

Bảo trì dự đoán là phương pháp bảo trì dựa trên dữ liệu vận hành thực tế của thiết bị nhằm phát hiện sớm các dấu hiệu bất thường trước khi xảy ra hư hỏng nghiêm trọng. Khác với bảo trì khắc phục, chỉ sửa chữa khi máy đã hỏng, hoặc bảo trì định kỳ theo lịch cố định, bảo trì dự đoán cho phép đánh giá tình trạng máy dựa trên các thông số đo được trong quá trình vận hành.

Trong các hệ thống máy công nghiệp, những sự cố như mất cân bằng cơ khí, lệch trục, mòn ổ bi, lỏng chi tiết, quá tải hoặc tăng ma sát thường làm thay đổi các đại lượng vật lý đặc trưng của máy. Các thay đổi này có thể được phản ánh thông qua độ rung, nhiệt độ, độ ẩm môi trường, dòng điện tiêu thụ hoặc công suất làm việc. Khi máy hoạt động bình thường, các thông số này thường dao động trong một khoảng ổn định. Ngược lại, khi xuất hiện bất thường, giá trị đo có thể tăng cao, dao động mạnh hoặc thay đổi theo xu hướng bất thường.

Giám sát tình trạng máy là quá trình thu thập, xử lý và theo dõi liên tục các thông số trên để đánh giá trạng thái vận hành của thiết bị. Trong phạm vi đồ án này, hệ thống sử dụng nút cảm biến không dây để thu thập dữ liệu rung động từ cảm biến MPU6050, nhiệt độ và độ ẩm từ cảm biến AHT30. Dữ liệu sau đó được xử lý sơ bộ tại ESP32 và truyền về phần mềm giám sát trung tâm để hiển thị theo thời gian thực.

Một cơ sở quan trọng trong giám sát tình trạng máy là xây dựng giá trị chuẩn, hay còn gọi là baseline. Đây là trạng thái tham chiếu khi máy hoạt động bình thường. Khi dữ liệu đo được lệch nhiều so với baseline hoặc vượt quá ngưỡng cho phép, hệ thống có thể đưa ra cảnh báo để người vận hành kiểm tra và xử lý kịp thời.

Nhờ đó, ta thấy rằng bảo trì dự đoán giúp giảm nguy cơ dừng máy đột ngột, hạn chế chi phí sửa chữa và nâng cao hiệu quả vận hành thiết bị. Đây cũng là cơ sở lý thuyết quan trọng để xây dựng hệ thống giám sát bảo trì máy móc công nghiệp trong đồ án này.

2.2. Rung động cơ khí và phân tích phổ FFT

Rung động cơ khí là một trong những tín hiệu quan trọng dùng để đánh giá tình trạng làm việc của máy quay, động cơ, quạt, bơm hoặc các cụm truyền động. Khi máy vận hành bình thường, tín hiệu rung thường có biên độ nhỏ và phân bố ổn định quanh một mức nhất định. Ngược lại, khi xuất hiện các hiện tượng như mất cân bằng động, lệch trục, lỏng cơ khí, mòn ổ bi hoặc va đập bất thường, biên độ rung có thể tăng lên rõ rệt hoặc xuất hiện các thành phần tần số đặc trưng.

Trong đồ án này, cảm biến MPU6050 được sử dụng để thu thập dữ liệu gia tốc rung động theo ba trục X, Y và Z. Tín hiệu thu được ban đầu nằm trong miền thời gian, tức là thể hiện sự thay đổi của gia tốc theo từng thời điểm lấy mẫu. Dữ liệu này giúp quan sát trực tiếp mức dao động của thiết bị, tuy nhiên chưa thể hiện rõ các thành phần tần số ẩn bên trong tín hiệu rung.

Để đánh giá đặc điểm rung động rõ hơn, hệ thống sử dụng phương pháp phân tích phổ FFT. FFT là thuật toán biến đổi nhanh Fourier, có nhiệm vụ chuyển tín hiệu từ miền thời gian sang miền tần số. Nhờ đó, hệ thống có thể xác định được tín hiệu rung tập trung mạnh tại những dải tần nào. Đây là cơ sở quan trọng để phát hiện các dấu hiệu

bất thường của máy móc, vì mỗi dạng lỗi cơ khí thường tạo ra sự thay đổi khác nhau trong phổ tần số.

Trong quá trình xử lý, ESP32 lấy mẫu dữ liệu từ cảm biến với tần số xác định, sau đó gom đủ một số lượng mẫu nhất định để thực hiện FFT. Theo nguyên lý Nyquist, nếu tần số lấy mẫu là 1000 Hz thì dải tần số có thể quan sát được tối đa là 500 Hz. Tuy nhiên, để giảm dung lượng dữ liệu truyền đi và phù hợp với yêu cầu giám sát thời gian thực, hệ thống chỉ trích xuất một phần phổ tần số quan trọng, tập trung vào vùng tần số thấp và trung bình, nơi thường xuất hiện nhiều dấu hiệu bất thường cơ khí.

Bên cạnh phân tích FFT, giá trị RMS của tín hiệu gia tốc cũng được sử dụng để đánh giá mức năng lượng rung động tổng thể. Khi giá trị RMS tăng cao so với trạng thái bình thường, điều đó cho thấy máy có thể đang rung mạnh hơn và cần được kiểm tra. Việc kết hợp giữa RMS và phổ FFT giúp hệ thống vừa đánh giá nhanh mức độ rung, vừa có cơ sở phân tích sâu hơn về nguyên nhân gây bất thường.

2.3. Edge Computing trên ESP32

Edge Computing là hướng xử lý dữ liệu ngay tại thiết bị biên, thay vì gửi toàn bộ dữ liệu thô về máy chủ trung tâm. Trong hệ thống giám sát bảo trì máy móc, cách tiếp cận này giúp giảm dung lượng truyền dữ liệu, rút ngắn thời gian phản hồi và tăng khả năng hoạt động độc lập của thiết bị khi kết nối mạng không ổn định.

Trong đồ án này, ESP32 được sử dụng làm bộ xử lý trung tâm của nút cảm biến. Vì điều khiển này có tích hợp Wi-Fi, Bluetooth/BLE và có khả năng xử lý đủ tốt cho các tác vụ IoT như đọc cảm biến, lọc dữ liệu, tính toán giá trị đặc trưng và truyền dữ liệu không dây. Nhờ đó, ESP32 phù hợp với yêu cầu xây dựng một trạm cảm biến nhỏ gọn, chi phí hợp lý và dễ triển khai trong môi trường thực nghiệm.

Tại thiết bị biên, ESP32 thực hiện thu thập dữ liệu từ các cảm biến như MPU6050 và AHT30. Dữ liệu sau khi đọc về không được truyền đi ngay lập tức ở dạng thô, mà được xử lý sơ bộ để tạo ra các thông tin có ý nghĩa hơn, chẳng hạn như giá trị rung động tổng hợp, nhiệt độ, độ ẩm và dữ liệu phổ FFT. Việc xử lý trước tại ESP32 giúp phần mềm trung tâm không phải đảm nhiệm toàn bộ quá trình tính toán, từ đó giảm tải cho máy tính giám sát.

Ngoài ra, ESP32 còn có thể thực hiện các thao tác như kiểm tra ngưỡng, phát hiện trạng thái bất thường và đóng gói dữ liệu theo định dạng phù hợp trước khi truyền đi. Điều này giúp hệ thống phản ứng nhanh hơn trong các tình huống cần cảnh báo, đồng thời làm cho cấu trúc truyền thông giữa nút cảm biến và máy tính trung tâm trở nên gọn nhẹ hơn.

Như vậy, việc ứng dụng Edge Computing trên ESP32 giúp hệ thống giám sát trong đồ án có tính linh hoạt cao hơn. Thiết bị không chỉ đóng vai trò thu thập dữ liệu, mà còn tham gia xử lý, rút gọn và chuẩn hóa dữ liệu trước khi gửi về giao diện giám sát. Đây là nền tảng quan trọng để phát triển hệ thống theo hướng IIoT và bảo trì dự đoán trong tương lai.

2.4. Truyền thông Wi-Fi/TCP Socket và định dạng JSON

Trong hệ thống giám sát, truyền thông không dây đóng vai trò kết nối giữa nút cảm biến và phần mềm giám sát trung tâm. Sau khi ESP32 thu thập và xử lý dữ liệu từ cảm biến, dữ liệu cần được gửi về máy tính để hiển thị, lưu trữ và phục vụ quá trình đánh giá tình trạng máy. Vì vậy, hệ thống sử dụng Wi-Fi làm phương thức truyền dữ

liệu chính do dễ triển khai, không cần dây tín hiệu phức tạp và phù hợp với mô hình giám sát trong phạm vi phòng thí nghiệm hoặc nhà xưởng nhỏ.

Trong đồ án này, giao tiếp giữa ESP32 và phần mềm máy tính được thực hiện thông qua TCP Socket. Đây là cơ chế truyền thông cho phép hai thiết bị trao đổi dữ liệu trực tiếp trong cùng một mạng. ESP32 đóng vai trò gửi dữ liệu từ nút cảm biến, còn phần mềm C# WinForms trên máy tính đóng vai trò tiếp nhận và xử lý dữ liệu. So với việc sử dụng Web Server, phương pháp TCP Socket giúp hệ thống gọn hơn, giảm số thành phần trung gian và phù hợp với sản phẩm thực nghiệm cần truyền dữ liệu theo thời gian thực.

Dữ liệu gửi đi được đóng gói theo định dạng JSON. JSON là định dạng dữ liệu dạng văn bản, có cấu trúc rõ ràng theo từng cặp tên và giá trị. Nhờ đó, phần mềm trung tâm có thể dễ dàng tách các trường dữ liệu như nhiệt độ, độ ẩm, gia tốc rung động, giá trị FFT hoặc trạng thái cảnh báo. Việc sử dụng JSON cũng giúp hệ thống dễ mở rộng, vì khi cần bổ sung thêm cảm biến hoặc thêm thông số đo mới, chỉ cần thêm trường dữ liệu tương ứng trong gói tin.

Một gói dữ liệu truyền từ ESP32 về máy tính có thể bao gồm các thông tin như thời gian đo, nhiệt độ, độ ẩm, dữ liệu rung động ba trục, dữ liệu phổ FFT và chế độ đánh thức của thiết bị. Sau khi nhận được gói JSON, phần mềm C# sẽ giải mã dữ liệu, phân loại theo từng nhóm thông số và cập nhật lên các biểu đồ giám sát.

Tóm lại, việc kết hợp Wi-Fi, TCP Socket và JSON giúp hệ thống truyền dữ liệu linh hoạt, dễ xử lý và phù hợp với yêu cầu giám sát thời gian thực. Cấu trúc này cũng tạo nền tảng để mở rộng sang các giao thức IoT chuyên dụng hơn như MQTT trong các phiên bản phát triển tiếp theo.

2.5. Giao diện giám sát C# WinForms và DevExpress

Phần mềm giám sát trung tâm được xây dựng trên nền tảng C# WinForms, kết hợp với thư viện giao diện DevExpress để trực quan hóa dữ liệu thu thập từ nút cảm biến. Đây là lựa chọn phù hợp với sản phẩm thực tế của đồ án vì cho phép xây dựng nhanh một giao diện desktop ổn định, dễ thao tác và không cần triển khai thêm Web Server phức tạp.

C# WinForms đảm nhiệm vai trò tạo giao diện người dùng, quản lý kết nối TCP Socket, tiếp nhận chuỗi dữ liệu JSON từ ESP32 và xử lý dữ liệu trước khi hiển thị. Khi có gói tin gửi về, chương trình sẽ tách các trường thông tin như nhiệt độ, độ ẩm, dữ liệu rung động và phổ FFT. Sau đó, dữ liệu được phân loại để đưa vào từng khu vực hiển thị tương ứng trên Dashboard.

DevExpress được sử dụng để thiết kế các biểu đồ trực quan, giúp người vận hành dễ theo dõi trạng thái của thiết bị. Các thành phần giao diện có thể bao gồm biểu đồ nhiệt độ – độ ẩm theo thời gian, biểu đồ rung động ba trục, biểu đồ phổ tần số FFT và các khu vực hiển thị trạng thái kết nối. Nhờ cách trình bày bằng đồ thị, người dùng có thể quan sát sự thay đổi của dữ liệu rõ ràng hơn so với việc chỉ hiển thị giá trị số.

Một ưu điểm của giao diện desktop là khả năng hoạt động trực tiếp trên máy tính giám sát trong cùng mạng Wi-Fi với ESP32. Điều này giúp quá trình thử nghiệm sản phẩm thuận tiện hơn, đặc biệt trong giai đoạn đồ án môn học. Người dùng chỉ cần khởi chạy phần mềm, kết nối với nút cảm biến và quan sát dữ liệu thời gian thực mà không cần cấu hình hệ thống máy chủ riêng.

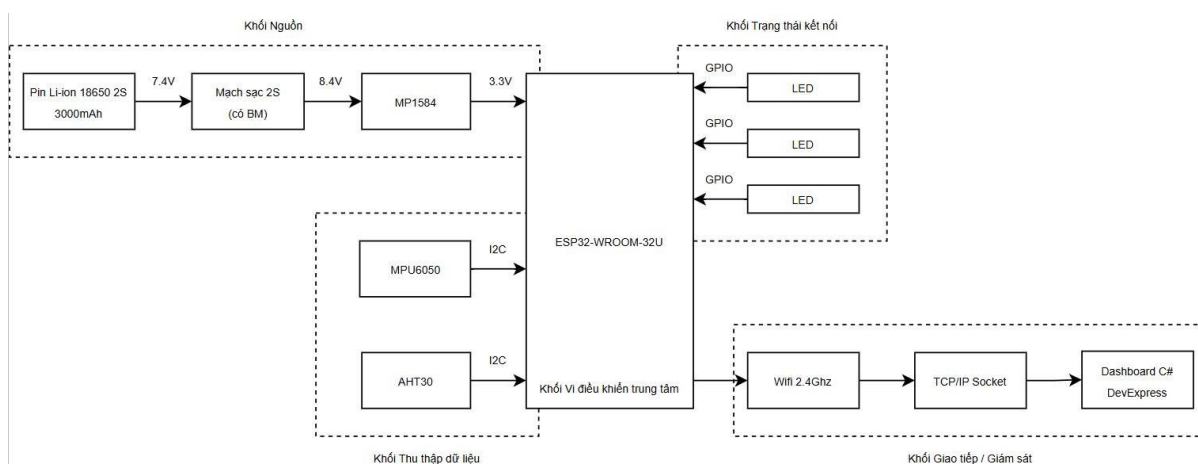
Việc xây dựng phần mềm giám sát bằng C# WinForms và DevExpress giúp sản phẩm có giao diện trực quan, dễ sử dụng và phù hợp với yêu cầu minh họa nguyên lý bảo trì dự đoán. Đây cũng là nền tảng để mở rộng thêm các chức năng như lưu lịch sử dữ liệu, cảnh báo vượt ngưỡng, xuất báo cáo hoặc kết nối nhiều nút cảm biến trong các phiên bản tiếp theo.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng quát của sản phẩm thực tế

Phần cứng của đề tài được thiết kế theo hướng nhỏ gọn, dễ lắp đặt và phù hợp với mô hình giám sát tình trạng máy móc công nghiệp ở quy mô thực nghiệm. Thiết bị cần thu thập được các thông số phản ánh trạng thái vận hành của máy, bao gồm rung động, nhiệt độ và độ ẩm môi trường. Các dữ liệu này được xử lý tại nút cảm biến, sau đó truyền không dây về phần mềm giám sát trung tâm để hiển thị và hỗ trợ đánh giá tình trạng thiết bị.

Về mặt cấu trúc, phần cứng được chia thành các khối chức năng chính gồm: khối cảm biến, khối xử lý trung tâm, khối nguồn, khối truyền thông không dây và khối phần mềm giám sát. Trong đó, khối cảm biến đảm nhiệm việc thu thập dữ liệu từ môi trường và máy móc; khối xử lý trung tâm thực hiện đọc dữ liệu, xử lý sơ bộ và đóng gói thông tin; khối truyền thông gửi dữ liệu về máy tính thông qua Wi-Fi; phần mềm giám sát tiếp nhận dữ liệu và hiển thị dưới dạng biểu đồ trực quan.

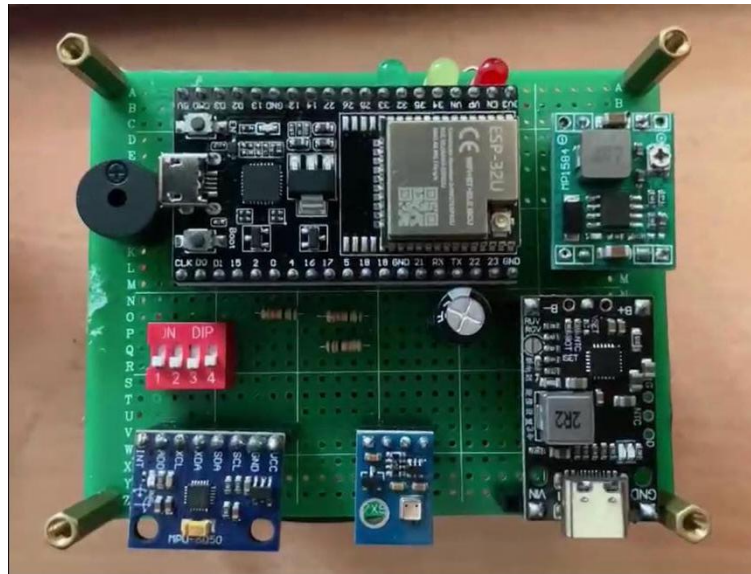


Hình 3.1. Sơ đồ khối kiến trúc phần cứng của hệ thống

Yêu cầu đầu tiên của phần cứng là đảm bảo khả năng đo lường ổn định. Cảm biến MPU6050 được sử dụng để thu thập tín hiệu rung động theo ba trục, phục vụ việc đánh giá dao động cơ khí của máy. Cảm biến AHT30 được dùng để đo nhiệt độ và độ ẩm môi trường xung quanh thiết bị. Các cảm biến cần được bố trí hợp lý nhằm hạn chế sai lệch do rung lắp đặt, nhiễu từ linh kiện khác hoặc tác động nhiệt cục bộ.

Khối xử lý trung tâm sử dụng ESP32 vì vi điều khiển này tích hợp sẵn Wi-Fi, có kích thước nhỏ gọn và đáp ứng được các tác vụ thu thập dữ liệu, xử lý tín hiệu, tính toán đặc trưng rung động và truyền dữ liệu không dây. Việc xử lý một phần dữ liệu ngay tại nút cảm biến giúp giảm tải cho phần mềm trung tâm, đồng thời làm cho gói dữ liệu truyền đi gọn hơn, phù hợp với yêu cầu giám sát thời gian thực.

Về kết cấu sản phẩm, toàn bộ mạch điện, nguồn nuôi và anten được bố trí trong một cụm thiết bị độc lập. Thiết bị được định hướng lắp đặt linh hoạt trên bề mặt kim loại của máy móc bằng nam châm, giúp quá trình thử nghiệm thuận tiện hơn mà không cần khoan, bắt vít hoặc thay đổi kết cấu cơ khí của máy. Cách thiết kế này phù hợp với mục tiêu tạo ra một nút cảm biến có thể tháo lắp nhanh và sử dụng cho nhiều vị trí khảo sát khác nhau.



Hình 3.2. Hình ảnh tổng thể sản phẩm thực tế sau khi lắp ráp hoàn chỉnh

Nguồn cung cấp cho thiết bị cần đảm bảo điện áp ổn định cho ESP32 và các cảm biến trong suốt quá trình hoạt động. Do sản phẩm được thiết kế theo dạng nút cảm biến không dây, khối nguồn cần đáp ứng yêu cầu gọn nhẹ, đủ thời gian vận hành và phù hợp với các chế độ tiết kiệm năng lượng. Ngoài ra, việc bố trí anten cũng cần được quan tâm để duy trì kết nối Wi-Fi ổn định khi thiết bị đặt gần các bề mặt kim loại hoặc trong môi trường có nhiều vật cản.

3.2. Lựa chọn phần cứng và danh sách linh kiện

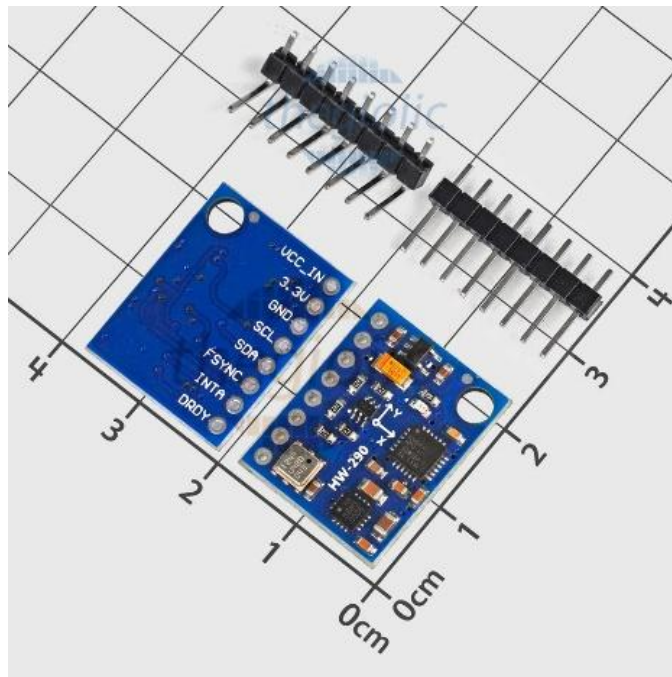
Phần cứng của sản phẩm được lựa chọn theo các tiêu chí: nhỏ gọn, dễ tích hợp, chi phí phù hợp, có khả năng hoạt động bằng nguồn pin và đáp ứng được yêu cầu thu thập dữ liệu phục vụ giám sát tình trạng máy móc. Do sản phẩm được triển khai dưới dạng nút cảm biến không dây, các linh kiện cần đảm bảo khả năng đo lường ổn định, truyền dữ liệu qua Wi-Fi và thuận tiện khi bố trí trong vỏ hộp.

Thành phần xử lý trung tâm của hệ thống là ESP32-WROOM-32U hoặc board ESP32 có hỗ trợ anten ngoài. ESP32 được lựa chọn vì tích hợp Wi-Fi, có khả năng xử lý dữ liệu cảm biến, đóng gói dữ liệu JSON và truyền thông không dây về phần mềm giám sát. Phiên bản có anten ngoài giúp cải thiện chất lượng kết nối khi thiết bị được đặt gần bề mặt kim loại hoặc bên trong vỏ hộp bảo vệ.



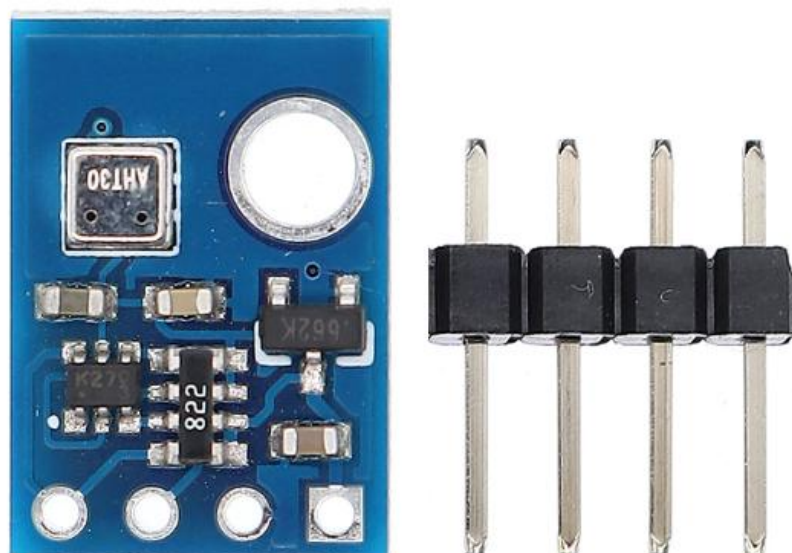
Hình 3.3. Board ESP32 sử dụng trong nút cảm biến

Cảm biến MPU6050 được sử dụng để thu thập dữ liệu rung động theo ba trục X, Y và Z. Đây là linh kiện quan trọng trong hệ thống vì dữ liệu gia tốc từ cảm biến được dùng để đánh giá dao động cơ khí của máy. Từ dữ liệu thu được, ESP32 có thể tính độ lớn rung động tổng hợp, xác định trạng thái cảnh báo và trích xuất tần số đặc trưng phục vụ quá trình giám sát.



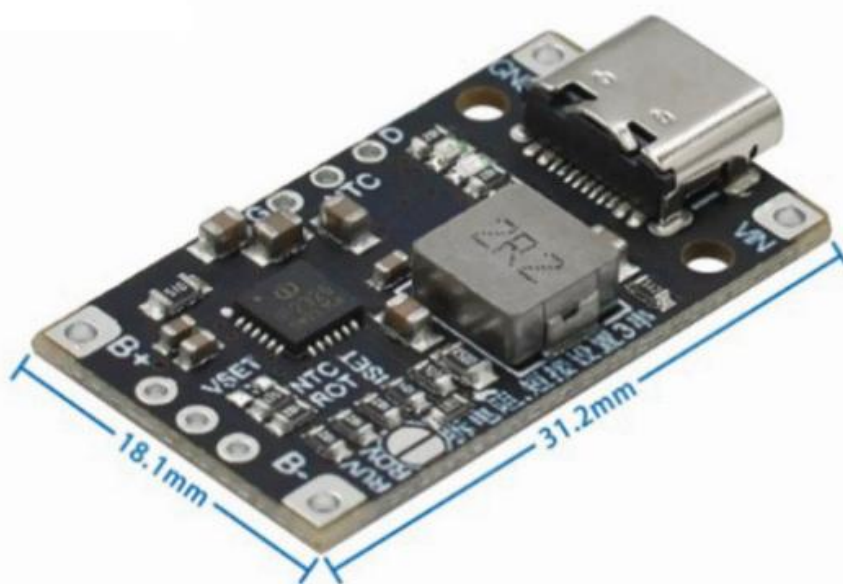
Hình 3.4. Cảm biến rung động MPU6050

Cảm biến AHT30 được dùng để đo nhiệt độ và độ ẩm môi trường xung quanh thiết bị. Hai thông số này giúp bổ sung thêm thông tin về điều kiện vận hành của cụm máy, đặc biệt trong các trường hợp môi trường có nhiệt độ cao hoặc độ ẩm lớn. Dữ liệu từ AHT30 cũng được gửi về phần mềm giám sát cùng với dữ liệu rung động để người vận hành theo dõi tổng thể trạng thái làm việc của thiết bị.



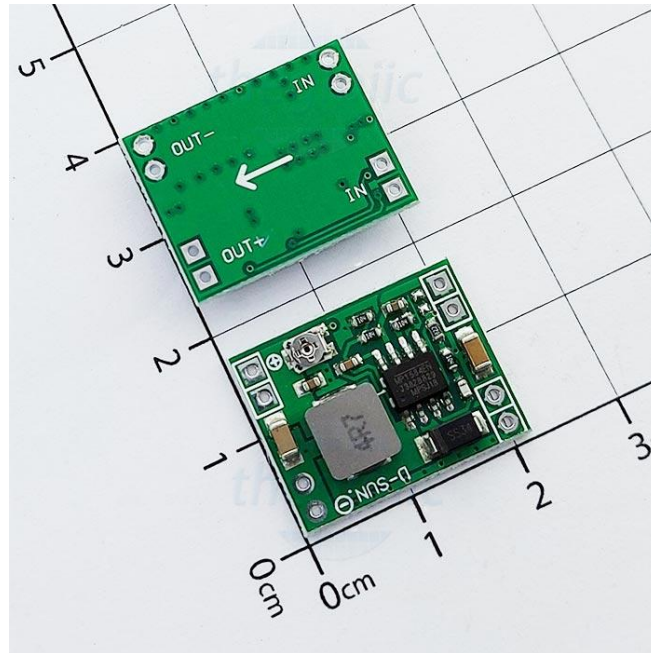
Hình 3.5. Cảm biến nhiệt độ và độ ẩm AHT30

Khối nguồn của sản phẩm gồm pin Lithium 18650, mạch bảo vệ pin BMS và mạch ổn áp 3,3 V. Cụm pin giúp nút cảm biến hoạt động độc lập, không phụ thuộc vào nguồn cấp cố định trong quá trình thử nghiệm. Mạch BMS có nhiệm vụ bảo vệ pin khi sạc, xả hoặc khi xảy ra sự cố quá dòng. Mạch ổn áp cung cấp điện áp phù hợp cho ESP32 và các cảm biến, giúp hệ thống hoạt động ổn định hơn.



Hình 3.6. Khối nguồn sử dụng cho nút cảm biến

Mạch giảm áp MP1584EN 3A được sử dụng để hạ điện áp từ cụm pin 2S xuống mức điện áp phù hợp cho ESP32 và các cảm biến. Do cụm pin gồm hai cell 18650 mắc nối tiếp có điện áp danh định khoảng 7,4 V, trong khi ESP32 và các cảm biến hoạt động chủ yếu ở mức 3,3 V, cần có một mạch hạ áp ổn định để cấp nguồn cho toàn hệ thống. MP1584EN là module giảm áp xung có kích thước nhỏ gọn, hiệu suất cao hơn so với ổn áp tuyến tính khi dùng với nguồn pin 2S, đồng thời có khả năng cấp dòng tối đa 3 A, đáp ứng tốt các xung dòng ngắn khi ESP32 truyền dữ liệu qua Wi-Fi. Việc sử dụng mạch giảm áp này giúp tăng hiệu quả sử dụng năng lượng, giảm hao phí nhiệt và hỗ trợ kéo dài thời lượng hoạt động của thiết bị khi dùng pin.



Hình 3.7. Mạch giảm áp MP1584EN 3A sử dụng trong khối nguồn của nút cảm biến

Ngoài các linh kiện điện tử chính, sản phẩm còn sử dụng anten ngoài, vỏ mica, đế nam châm, dây nối, công tắc, đầu nối và các chi tiết cố định cơ khí. Anten ngoài giúp tăng độ ổn định khi truyền dữ liệu Wi-Fi. Vỏ mica có nhiệm vụ bảo vệ bo mạch, pin và dây kết nối, đồng thời giúp quan sát được bố trí linh kiện bên trong trong quá trình kiểm tra và trình bày sản phẩm. Đế nam châm giúp thiết bị có thể gắn nhanh lên bề mặt kim loại của máy, thuận tiện cho quá trình thử nghiệm mà không cần khoan hoặc thay đổi kết cấu cơ khí của thiết bị cần giám sát.

Các linh kiện được lựa chọn và sắp xếp theo từng nhóm chức năng của hệ thống, bao gồm khối thu thập dữ liệu, khối xử lý trung tâm, khối truyền thông, khối nguồn và khối gá lắp cơ khí. Cách phân loại này giúp làm rõ vai trò của từng thành phần trong sản phẩm, đồng thời thuận tiện cho quá trình lắp ráp, kiểm tra và mở rộng hệ thống trong các phiên bản sau.

Bảng 3.1. Danh sách linh kiện sử dụng trong hệ thống

STT	Tên linh kiện	Số lượng	Thông số kỹ thuật	Vai trò
1	ESP32-WROOM-32U	1	Dual-core 240 MHz, Wi-Fi, anten ngoài SMA.	Vi điều khiển trung tâm, đọc cảm biến, xử lý dữ liệu và truyền Wi-Fi.
2	Module MPU6050	1	Cảm biến gia tốc 3 trục, giao tiếp I2C.	Thu thập rung động phục vụ phân tích RMS/FFT và phát hiện dao động bất thường.
3	Module AHT30	1	Cảm biến nhiệt độ, độ ẩm, giao tiếp I2C.	Theo dõi nhiệt độ và độ ẩm môi trường quanh thiết bị.
4	Cell Pin Li-Ion 18650	2	3.7 V, dung lượng khoảng 3000 mAh/cell.	Cấp nguồn độc lập cho nút cảm biến không dây.
5	Mạch sạc pin 2S Type-C tích hợp cân bằng	1	Sạc 2 cell Li-Ion nối tiếp, đầu vào Type-C 5 V, đầu ra 8.4 V.	Sạc và cân bằng điện áp cho cụm pin 2S.

STT	Tên linh kiện	Số lượng	Thông số kỹ thuật	Vai trò
6	Mạch giảm áp MP1584EN 3A	1	Nguồn vào 4.75–28 VDC, hiệu suất cao, chỉnh được điện áp ra.	Hạ áp từ cụm pin xuống 3.3 V cấp cho ESP32 và cảm biến.
7	Tụ hóa lọc nguồn	1	470 uF, điện áp chịu đựng 16 V hoặc 25 V.	Lọc nhiễu nguồn và giảm dao động điện áp đầu ra.
8	Điện trở	2	22 kΩ và 10 kΩ, công suất 1/4 W.	Tạo cầu phân áp đo điện áp pin bằng ADC của ESP32.
9	Vỏ mica trong suốt	1	Mica trong suốt, cắt theo kích thước sản phẩm	Bảo vệ mạch, pin và tăng độ bền cơ khí của thiết bị.
10	Đế nam châm đất hiếm	5	Nam châm vĩnh cửu lực hút mạnh, kích thước D15-5 mm.	Gắn nhanh thiết bị lên bề mặt kim loại của máy cần giám sát.
11	Anten Wi-Fi SMA 2.4 GHz + cáp U.FL sang SMA	1	Anten 2.4 GHz, đầu nối SMA, cáp chuyển U.FL.	Đưa Anten Wi-Fi ra ngoài vỏ hộp để tăng ổn định kết nối.
12	LED chỉ thị trạng thái	3	LED 5 mm, màu đỏ, vàng, xanh.	Hiển thị trạng thái kết nối, truyền dữ liệu và lỗi/hết pin.

3.3. Tính toán thời lượng sử dụng pin

Nút cảm biến được thiết kế để hoạt động độc lập bằng pin, do đó thời lượng sử dụng là một thông số quan trọng cần được ước lượng trong quá trình thiết kế. Trong hệ thống này, thiết bị không duy trì trạng thái hoạt động liên tục mà sử dụng cơ chế Deep Sleep. ESP32 chỉ thức dậy theo chu kỳ định kỳ hoặc khi có rung động vượt ngưỡng, sau đó thực hiện đọc cảm biến, xử lý dữ liệu, truyền gói tin về máy tính giám sát và quay lại trạng thái ngủ sâu.

Theo tài liệu kỹ thuật của ESP32, ở chế độ Deep Sleep, khi chỉ duy trì RTC timer và RTC memory, dòng tiêu thụ của chip có thể ở mức rất thấp; tuy nhiên trong sản phẩm thực tế, dòng ngủ sâu của toàn mạch còn bao gồm dòng rò của cảm biến, mạch BMS, mạch hạ áp, LED hoặc các linh kiện phụ trợ. Vì vậy, phần tính toán dưới đây không dùng dòng Deep Sleep lý tưởng của riêng chip ESP32, mà sử dụng dòng ngủ sâu ước lượng của toàn mạch để kết quả thực tế hơn.

3.3.1. Thông số pin và giả thiết tính toán

Cụm pin sử dụng trong thiết bị gồm hai viên pin Lithium 18650 mắc nối tiếp. Mỗi viên có điện áp danh định 3,7 V và dung lượng 3000 mAh. Khi mắc nối tiếp, điện áp danh định của cụm pin tăng lên 7,4 V, nhưng dung lượng theo Ah vẫn giữ ở mức 3000 mAh.

Bảng 3.2. Thông số cụm pin sử dụng trong tính toán

Thông số	Giá trị sử dụng trong tính toán
Số cell pin	2 viên 18650 mắc nối tiếp
Điện áp danh định mỗi cell	3,7 V
Dung lượng mỗi cell	3000 mAh = 3 Ah

Điện áp danh định của cụm pin	7,4 V
Dung lượng danh định của cụm pin	3000 mAh
Hiệu suất mạch hạ áp dùng để ước lượng	85%

$$E_{\text{pin}} = U_{\text{pin}} \times C_{\text{pin}} = 7,4 \times 3 = 22,2 \text{ Wh}$$

Trong đó, E_{pin} là năng lượng danh định của cụm pin, U_{pin} là điện áp danh định của cụm pin và C_{pin} là dung lượng pin tính theo Ah. Do thiết bị sử dụng mạch hạ áp để tạo mức 3,3 V cho ESP32 và cảm biến, năng lượng hữu ích ở phía tải được ước lượng với hiệu suất chuyển đổi 85%:

$$E_{\text{out}} = E_{\text{pin}} \times \eta = 22,2 \times 0,85 = 18,87 \text{ Wh}$$

3.3.2. Thời gian hoạt động trong một ngày

Thiết bị được cấu hình đánh thức định kỳ 5 phút/lần. Trong một ngày có 24 giờ, số lần đánh thức định kỳ là:

$$N_{\text{periodic}} = 24 \times 60 / 5 = 288 \text{ lần/ngày}$$

Ngoài các lần đánh thức định kỳ, thiết bị có thể được đánh thức khẩn cấp khi cảm biến rung phát hiện vượt ngưỡng. Trong phép tính này, giả thiết có thêm 5 lần đánh thức khẩn cấp trong một ngày. Từ quá trình phân rã thời gian firmware, một lần đánh thức khẩn cấp gồm các giai đoạn: khởi động và kết nối Wi-Fi, lấy mẫu dữ liệu, xử lý FFT, gửi JSON qua TCP Socket, chụp nhanh dữ liệu rung động và trở an toàn trước khi quay lại Deep Sleep. Tổng thời gian của một lần xử lý khẩn cấp được ước lượng khoảng 7,21 giây.

Tổng thời gian thiết bị ở trạng thái hoạt động trong một ngày được lấy là 591 giây. Giá trị này bao gồm các lần đánh thức định kỳ và 5 lần đánh thức khẩn cấp do rung. Phần thời gian còn lại trong ngày là thời gian thiết bị ở chế độ Deep Sleep:

$$T_{\text{sleep}} = 86400 - 591 = 85809 \text{ s}$$

Bảng 3.3. Giả thiết chu kỳ hoạt động và Deep Sleep của thiết bị

Thông số vận hành	Giá trị
Chu kỳ đánh thức định kỳ	5 phút/lần
Số lần đánh thức định kỳ trong 1 ngày	288 lần/ngày
Số lần đánh thức khẩn cấp giả định	5 lần/ngày
Thời gian xử lý của một lần khẩn cấp	khoảng 7,21 s
Tổng thời gian hoạt động trong 1 ngày	591 s
Tổng thời gian Deep Sleep trong 1 ngày	85809 s

3.3.3. Tính dòng tiêu thụ trung bình

Dòng tiêu thụ của thiết bị được chia thành hai trạng thái: trạng thái hoạt động và trạng thái Deep Sleep. Ở trạng thái hoạt động, ESP32 đọc cảm biến, xử lý dữ liệu và truyền thông Wi-Fi, nên dòng tiêu thụ cao hơn. Ở trạng thái Deep Sleep, phần lớn ngoại vi được tắt hoặc đưa về trạng thái tiêu thụ thấp.

Bảng 3.4. Thông số dòng điện sử dụng để tính I_{avg}

Đại lượng	Giá trị
Dòng khi hoạt động, I_{active}	79 mA
Dòng khi Deep Sleep toàn mạch, I_{sleep}	0,15 mA
Tổng thời gian hoạt động/ngày, T_{active}	591 s
Tổng thời gian Deep Sleep/ngày, T_{sleep}	85809 s
Tổng thời gian trong ngày, T_{total}	86400 s

Dòng điện tiêu thụ trung bình trong một ngày được tính theo công thức:

$$I_{avg} = (I_{active} \times T_{active} + I_{sleep} \times T_{sleep}) / T_{total}$$

Thay các giá trị đã chọn:

$$I_{avg} = (79 \times 591 + 0,15 \times 85809) / 86400$$

$$I_{avg} \approx 0,689 \text{ mA}$$

3.3.4. Ước lượng thời lượng pin

Nếu các giá trị dòng I_{active} và I_{sleep} được hiểu là dòng tiêu thụ ở phía tải 3,3 V, cách tính chặt chẽ nhất là tính theo năng lượng. Công suất trung bình của tải là:

$$P_{avg} = 3,3 \times 0,000689 \approx 0,00227 \text{ W}$$

Với năng lượng hữu ích sau mạch hạ áp là 18,87 Wh, thời lượng pin lý thuyết là:

$$t_{theory} = E_{out} / P_{avg} = 18,87 / 0,00227 \approx 8295 \text{ h}$$

$$t_{theory} \approx 345,6 \text{ ngày} \approx 11,5 \text{ tháng}$$

Tuy nhiên, kết quả trên là giá trị lý thuyết trong điều kiện thuận lợi. Trong thực tế, thời lượng có thể giảm do dung lượng thật của pin thấp hơn dung lượng danh định, dòng rò của BMS, hiệu suất mạch hạ áp thay đổi ở tải rất nhẹ, thời gian kết nối Wi-Fi kéo dài, số lần đánh thức khẩn cấp nhiều hơn giả thiết và nhiệt độ môi trường ảnh hưởng đến pin.

3.4. Thiết kế cơ khí và bố trí vỏ hộp

Vỏ hộp của nút cảm biến được lựa chọn theo hướng nhỏ gọn, chắc chắn và phù hợp với mục tiêu thử nghiệm sản phẩm ở quy mô phòng thí nghiệm. Việc đặt bo mạch, cụm pin và dây kết nối trong vỏ bảo vệ giúp hạn chế va chạm trực tiếp từ bên ngoài, đồng thời làm cho sản phẩm có hình thức hoàn chỉnh hơn khi lắp đặt lên mô hình máy móc hoặc thiết bị thử nghiệm.

Trong phiên bản sản phẩm hiện tại, vật liệu mica được sử dụng để chế tạo vỏ hộp. Mica có ưu điểm là nhẹ, dễ gia công, dễ cắt theo kích thước mong muốn và có tính thẩm mỹ tốt. Bên cạnh đó, đặc tính trong suốt của mica giúp người dùng quan sát được bố trí linh kiện bên trong, thuận tiện cho quá trình kiểm tra sản phẩm. So với vỏ kim loại, vỏ mica không gây chắn sóng Wi-Fi đáng kể, vì vậy phù hợp với thiết bị cảm biến không dây sử dụng ESP32.

Bên trong vỏ hộp, các thành phần chính được bố trí theo hướng tối ưu không gian và giảm rối dây. Bo mạch xử lý được đặt ở vị trí dễ quan sát và thuận tiện cho việc kiểm tra kết nối, nạp chương trình hoặc thay đổi linh kiện khi cần. Cụm pin và khối nguồn được cố định chắc chắn nhằm hạn chế rung lắc, tránh lỏng dây hoặc chập mạch trong quá trình thử nghiệm.

Đối với cảm biến MPU6050, vị trí lắp đặt cần đảm bảo khả năng truyền rung tốt từ thân máy đến cảm biến. Cảm biến phải được cố định chắc chắn để hạn chế sai lệch do lỏng cơ khí hoặc do rung phụ từ chính vỏ thiết bị. Nếu cảm biến không được cố định tốt, dữ liệu gia tốc thu được có thể không phản ánh đúng dao động thực tế của máy, làm ảnh hưởng đến kết quả phân tích rung động và phổ FFT.

Cảm biến AHT30 được bố trí ở vị trí thông thoáng vừa đủ để đo nhiệt độ và độ ẩm môi trường xung quanh cụm máy. Vị trí đặt cảm biến cần tránh tiếp xúc trực tiếp với nguồn nhiệt cục bộ hoặc bị che kín hoàn toàn, vì điều này có thể làm sai lệch kết quả đo so với điều kiện môi trường thực tế.

Khối nguồn, dây pin và các đầu nối bên trong vỏ mica được cố định để tránh chập mạch, đứt dây hoặc va đập khi thiết bị chịu rung động. Anten ngoài vẫn được sử dụng nhằm cải thiện độ ổn định của kết nối Wi-Fi và giúp việc bố trí thiết bị linh hoạt hơn khi đặt gần bề mặt kim loại của máy.

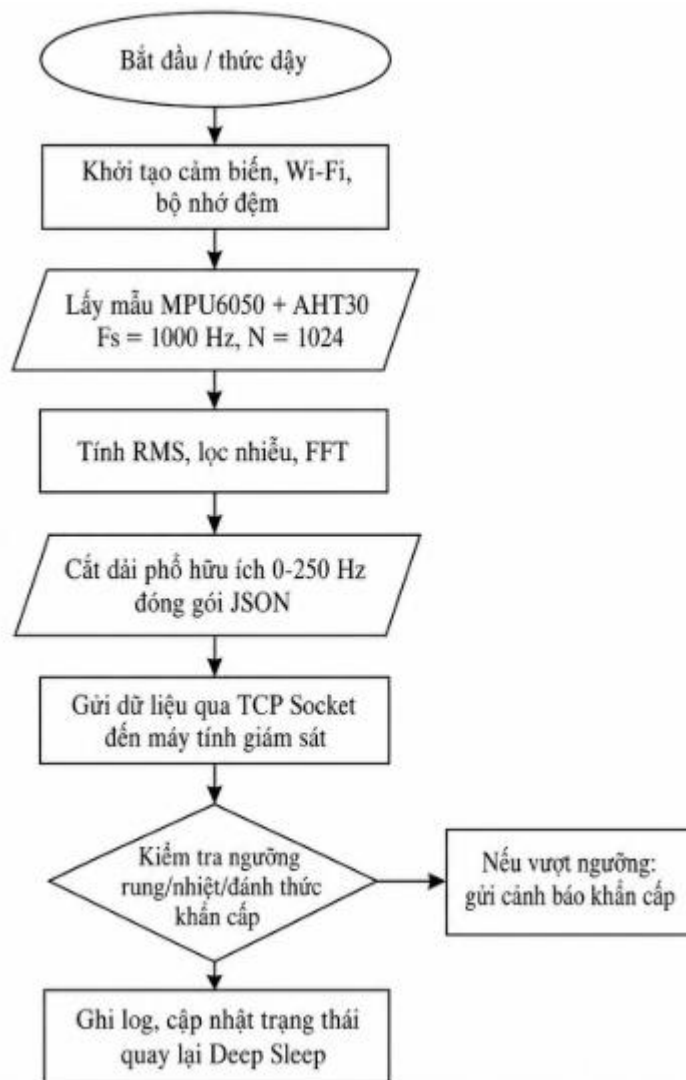
Tổng thể, thiết kế cơ khí của sản phẩm hướng đến ba yêu cầu chính: bảo vệ linh kiện, thuận tiện quan sát – kiểm tra và đảm bảo chất lượng tín hiệu đo. Việc sử dụng vỏ mica giúp sản phẩm có khối lượng nhẹ, dễ gia công, phù hợp với mô hình đồ án và thuận tiện cho quá trình thử nghiệm, trình bày sản phẩm.

3.5. Thiết kế phần mềm nhúng

Phần mềm nhúng trên ESP32 được xây dựng nhằm điều khiển toàn bộ quá trình hoạt động của nút cảm biến, từ khởi tạo hệ thống, thu thập dữ liệu, xử lý tín hiệu đến truyền dữ liệu về máy tính giám sát. Firmware được thiết kế theo hướng xử lý tại biên, nghĩa là một phần dữ liệu được tính toán trực tiếp trên ESP32 trước khi gửi đi. Cách tổ chức này giúp giảm dung lượng truyền, rút ngắn thời gian phản hồi và phù hợp với mô hình nút cảm biến không dây dùng trong giám sát tình trạng máy móc.

Quy trình tổng quát của firmware bắt đầu từ trạng thái khởi động hoặc được đánh thức từ chế độ tiết kiệm năng lượng. Sau đó, ESP32 tiến hành khởi tạo cảm biến, thiết

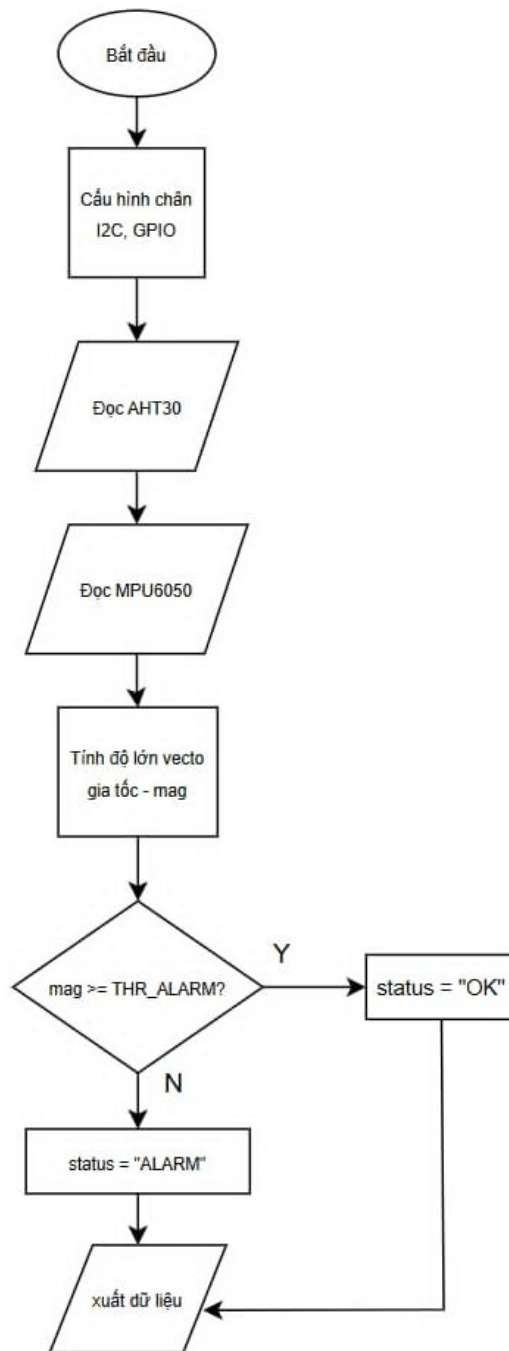
lập kết nối mạng, chuẩn bị bộ nhớ đệm và các biến xử lý dữ liệu. Khi hệ thống đã sẵn sàng, vi điều khiển thực hiện lấy mẫu dữ liệu cảm biến, xử lý tín hiệu rung động, đóng gói dữ liệu và gửi về phần mềm giám sát. Kết thúc một chu kỳ đo, thiết bị cập nhật trạng thái hoạt động và có thể quay lại chế độ Deep Sleep để tiết kiệm năng lượng.



Hình 3.8. Lưu đồ xử lý firmware ESP32

Đối với khối cảm biến, ESP32 thực hiện đọc dữ liệu từ cảm biến rung động MPU6050 và cảm biến nhiệt độ - độ ẩm AHT30. MPU6050 cung cấp dữ liệu gia tốc theo ba trục X, Y và Z, phục vụ việc đánh giá dao động cơ khí của máy. Trong khi đó, AHT30 cung cấp thông tin nhiệt độ và độ ẩm môi trường, giúp bổ sung thêm điều kiện vận hành xung quanh thiết bị. Các dữ liệu cảm biến sau khi thu thập được kiểm tra, lọc nhiễu và chuyển sang các giá trị đặc trưng cần thiết cho quá trình giám sát.

Với tín hiệu rung động, firmware tính giá trị RMS để phản ánh mức năng lượng rung tổng thể. Bên cạnh đó, thuật toán FFT được sử dụng để chuyển tín hiệu gia tốc từ miền thời gian sang miền tần số. Sau khi phân tích FFT, chương trình chỉ giữ lại dải tần số cần thiết để giảm kích thước gói dữ liệu truyền đi. Việc xử lý dữ liệu ngay tại ESP32 giúp phần mềm trung tâm không phải nhận toàn bộ dữ liệu thô, đồng thời hỗ trợ hiển thị phổ rung động thuận tiện hơn trên Dashboard.

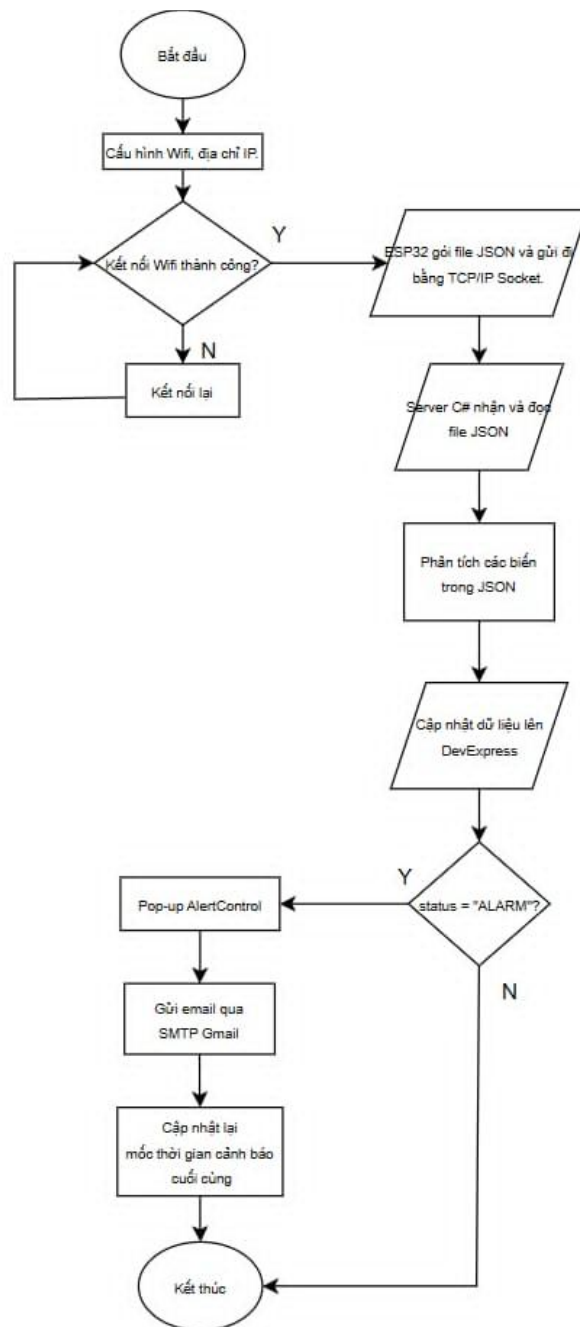


Hình 3.9. Lưu đồ xử lý cảm biến

Sau khi dữ liệu đã được xử lý, ESP32 đóng gói các trường thông tin cần thiết dưới dạng JSON. Gói dữ liệu có thể bao gồm mã thiết bị, nhiệt độ, độ ẩm, giá trị rung động, dữ liệu phổ FFT, chế độ đánh thức và trạng thái cảnh báo. Định dạng JSON được sử dụng vì có cấu trúc rõ ràng, dễ mở rộng và thuận tiện cho phần mềm giám sát tách dữ liệu sau khi nhận.

Khởi truyền thông đảm nhiệm việc kết nối nút cảm biến với phần mềm giám sát trung tâm thông qua Wi-Fi và giao thức TCP Socket. Sau khi kết nối vào cùng mạng Wi-Fi với máy tính giám sát, ESP32 sử dụng địa chỉ IP và cổng TCP đã cấu hình để gửi dữ liệu trực tiếp đến phần mềm C# WinForms. Khi có dữ liệu mới, firmware đóng gói các thông số đo và trạng thái thiết bị thành chuỗi JSON, sau đó gửi qua socket để phần mềm trung tâm tiếp nhận, phân tách và hiển thị trên giao diện. Nếu mất kết nối mạng

hoặc không gửi được dữ liệu, ESP32 sẽ thực hiện kết nối lại trước khi tiếp tục chu kỳ truyền dữ liệu tiếp theo.



Hình 3.10. Lưu đồ truyền thông Wifi TCP Socket

Về mặt nguyên lý, ESP32 kiểm tra trạng thái kết nối Wi-Fi, chuẩn bị gói dữ liệu JSON và gửi trực tiếp đến máy tính giám sát thông qua TCP Socket. Phần mềm C# WinForms đóng vai trò nhận dữ liệu tại địa chỉ IP và cổng TCP đã thiết lập, sau đó giải mã JSON để cập nhật các biểu đồ và trạng thái cảnh báo. Cách truyền trực tiếp này phù hợp với mô hình thử nghiệm của đề án vì cấu trúc đơn giản, dễ cấu hình và không cần triển khai thêm MQTT Broker hay máy chủ trung gian.

Tổng thể, phần mềm nhúng của hệ thống được tổ chức theo chu trình: khởi tạo, đọc cảm biến, xử lý dữ liệu, đóng gói JSON, truyền dữ liệu qua Wi-Fi/TCP Socket và quản lý trạng thái hoạt động. Cấu trúc này giúp nút cảm biến ESP32 hoạt động ổn định, đáp ứng yêu cầu giám sát rung động, nhiệt độ, độ ẩm và tạo nền tảng để mở rộng thêm

các chức năng cảnh báo, lưu trữ dữ liệu hoặc kết nối nhiều thiết bị trong các phiên bản tiếp theo.

3.6. Cấu trúc gói dữ liệu JSON

Sau khi ESP32 thu thập và xử lý dữ liệu từ các cảm biến, dữ liệu cần được đóng gói thành một định dạng thống nhất trước khi truyền về phần mềm giám sát trung tâm. Trong đề tài này, định dạng JSON được sử dụng để biểu diễn dữ liệu vì có cấu trúc rõ ràng, dễ đọc, dễ phân tách bằng phần mềm C# và thuận tiện khi cần mở rộng thêm trường dữ liệu trong các phiên bản sau.

JSON là định dạng dữ liệu dạng văn bản, tổ chức thông tin theo cặp “tên trường – giá trị”. Mỗi trường trong gói tin đảm nhiệm một ý nghĩa riêng, giúp phần mềm giám sát nhận biết dữ liệu đang thuộc nhóm nào và cần hiển thị ở khu vực nào trên giao diện. Cách tổ chức này phù hợp với hệ thống giám sát thời gian thực, vì dữ liệu từ nút cảm biến có thể được gửi liên tục theo từng chu kỳ đo hoặc theo từng sự kiện đánh thức.

Trong firmware, dữ liệu được chia thành hai nhóm chính. Nhóm thứ nhất là gói dữ liệu rung động, dùng để gửi các thông tin liên quan đến tín hiệu gia tốc, trạng thái cảnh báo và tần số đặc trưng. Nhóm thứ hai là gói trạng thái hệ thống, dùng để gửi các thông tin tổng quát như nhiệt độ, độ ẩm, điện áp pin, phần trăm pin và số lần thiết bị được đánh thức. Việc tách dữ liệu thành hai nhóm giúp gói tin gọn hơn, đồng thời giúp phần mềm trung tâm xử lý dữ liệu theo đúng mục đích hiển thị.

Gói dữ liệu rung động được tạo trong hàm `sendVibJson(...)`. Gói này chứa các trường như `type`, `boot`, `idx`, `wake`, `ax`, `ay`, `az`, `mag`, `status`, `temp`, `hum`, `vbat`, `pbat`, `totalWakes`, `motionWakes` và `peakFreq`. Trong đó, `ax`, `ay`, `az` là dữ liệu gia tốc theo ba trục; `mag` là độ lớn gia tốc tổng hợp; `status` thể hiện trạng thái cảnh báo của thiết bị; còn `peakFreq` là tần số đỉnh được tính từ dữ liệu rung động.

```
// 1. gói JSON về các dữ liệu rung động
void sendVibJson(float ax, float ay, float az, float temp, float humi, const char* wakeReason, uint32_t sampleIdx) {
    float mag = sqrtf(ax*ax + ay*ay + az*az);
    const char* status = (mag >= THR_ALARM) ? "ALARM" : (mag >= THR_WARN) ? "WARN" : "OK";

    JsonDocument doc;
    doc["type"] = "vib";
    doc["boot"] = bootCount;
    doc["idx"] = sampleIdx;
    doc["wake"] = wakeReason;
    doc["ax"] = serialized(String(ax, 3));
    doc["ay"] = serialized(String(ay, 3));
    doc["az"] = serialized(String(az, 3));
    doc["mag"] = serialized(String(mag, 3));
    doc["status"] = status;
    doc["temp"] = temp;
    doc["hum"] = humi;
    doc["vbat"] = serialized(String(batteryVoltage, 2));
    doc["pbat"] = serialized(String(batteryPercentage, 1));
    doc["totalWakes"] = totalWakes;
    doc["motionWakes"] = motionWakes;
    doc["peakFreq"] = serialized(String(calculatePeakFrequency(), 2));

    String output;
    serializeJson(doc, output);
    broadcastJson(output);
}
```

Hình 3.11. Đoạn chương trình tạo gói JSON dữ liệu rung động

Trong hàm `sendVibJson(...)`, độ lớn rung động tổng hợp được tính từ ba thành phần gia tốc theo các trục X, Y và Z. Dựa trên giá trị này, firmware phân loại trạng thái của thiết bị thành các mức OK, WARN hoặc ALARM. Cách phân loại này giúp phần mềm giám sát nhanh chóng nhận biết trạng thái vận hành mà không cần xử lý lại toàn bộ dữ liệu thô.

Bên cạnh gói rung động, firmware còn sử dụng hàm `sendStatusJson(...)` để tạo gói trạng thái hệ thống. Gói này không chứa dữ liệu gia tốc theo từng trục mà tập trung vào các thông tin tổng quát như số lần khởi động, tổng số lần thức dậy, số lần thức dậy do rung động, nhiệt độ, độ ẩm, điện áp pin và phần trăm pin. Nhờ đó, phần mềm giám sát vẫn có thể theo dõi tình trạng hoạt động của nút cảm biến ngay cả khi không có sự kiện rung động bất thường.

```
// 2. gói JSON về các dữ liệu trạng thái hệ thống, nhiệt độ, độ ẩm.
void sendStatusJson(float temp, float humi) {
    JsonDocument doc;
    doc["type"]      = "status";
    doc["boot"]      = bootCount;
    doc["totalWakes"] = totalWakes;
    doc["motionWakes"] = motionWakes;
    doc["temp"]      = temp;
    doc["hum"]       = humi;
    doc["vbat"]      = serialized(String(batteryVoltage, 2));
    doc["pbat"]      = serialized(String(batteryPercentage, 1));

    String output;
    serializeJson(doc, output);
    broadcastJson(output);
}
```

Hình 3.12. Đoạn chương trình tạo gói JSON trạng thái hệ thống

Khi phần mềm C# WinForms nhận được dữ liệu, chương trình sẽ kiểm tra trường `type` để xác định loại gói tin. Nếu `type` có giá trị `vib`, phần mềm cập nhật dữ liệu rung động, trạng thái cảnh báo và tần số đỉnh. Nếu `type` có giá trị `status`, phần mềm cập nhật thông tin trạng thái hệ thống như nhiệt độ, độ ẩm, pin và số lần đánh thức. Cách xử lý này giúp giao diện giám sát hiển thị dữ liệu rõ ràng và tránh nhầm lẫn giữa các nhóm thông tin.

Việc sử dụng cấu trúc JSON giúp hệ thống có tính linh hoạt cao. Khi cần bổ sung thêm cảm biến, thêm trạng thái cảnh báo hoặc mở rộng sang giao thức truyền thông khác, nội dung gói dữ liệu vẫn có thể được giữ nguyên và chỉ cần thêm các trường mới tương ứng. Đây là cơ sở quan trọng để hệ thống tiếp tục phát triển theo hướng giám sát nhiều thông số và nhiều nút cảm biến trong tương lai.

3.7. Thiết kế phần mềm giám sát trung tâm

Phần mềm giám sát trung tâm được xây dựng trên nền tảng C# WinForms, kết hợp với các thành phần biểu đồ của DevExpress để hiển thị dữ liệu theo thời gian thực. Phần mềm đóng vai trò tiếp nhận dữ liệu từ nút cảm biến ESP32, giải mã gói JSON, cập nhật các thông số đo và trực quan hóa trạng thái vận hành của thiết bị. Cách triển khai bằng ứng dụng desktop giúp quá trình thử nghiệm thuận tiện, không cần cấu hình thêm Web Server và phù hợp với quy mô của sản phẩm thực tế.

Ở phía truyền thông, phần mềm tạo một luồng xử lý nền để lắng nghe dữ liệu gửi về từ ESP32 thông qua TCP Socket. Khi nhận được chuỗi JSON, chương trình kiểm tra định dạng gói tin, tách các trường dữ liệu cần thiết và chuyển chúng vào các biến tương ứng. Việc xử lý dữ liệu ở luồng nền giúp giao diện không bị treo khi hệ thống liên tục nhận dữ liệu từ nút cảm biến.

Các dữ liệu sau khi giải mã được phân loại thành từng nhóm để hiển thị trên Dashboard. Nhóm dữ liệu môi trường gồm nhiệt độ và độ ẩm từ cảm biến AHT30. Nhóm

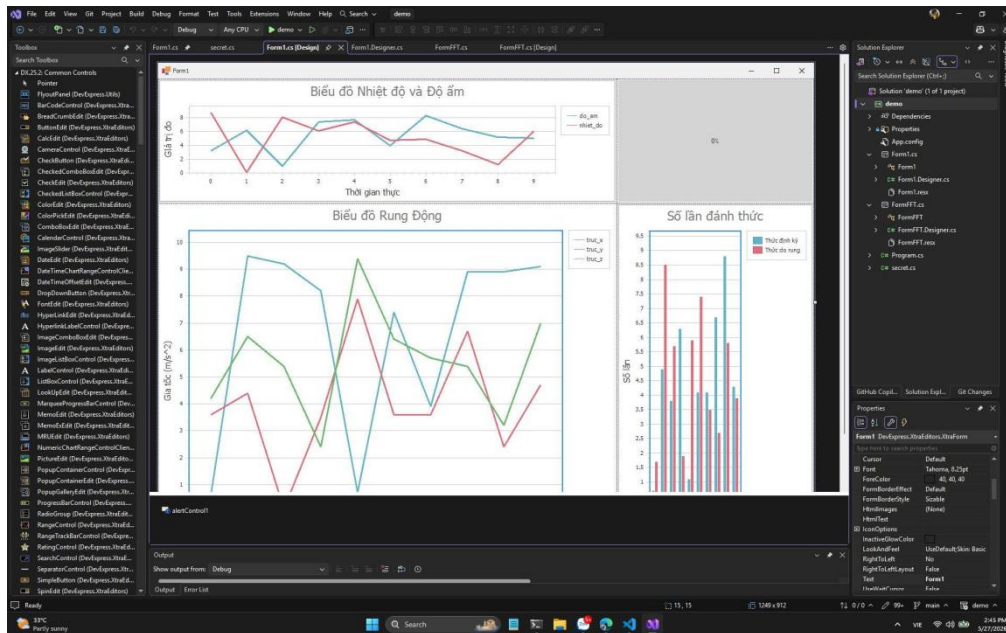
dữ liệu rung động gồm gia tốc theo các trục hoặc giá trị RMS từ MPU6050. Riêng dữ liệu FFT được đưa vào biểu đồ phổ tần số để người dùng quan sát sự thay đổi của các thành phần rung động trong dải tần đã chọn. Ngoài ra, phần mềm còn hiển thị trạng thái kết nối, thời gian nhận gói cuối, log dữ liệu và trạng thái cảnh báo.

Bảng 3.5. Các thành phần giao diện giám sát đề xuất

Khu vực giao diện	Nội dung hiển thị	Mục đích
Thông tin thiết bị	ID node, trạng thái kết nối, thời gian nhận gói cuối	Xác nhận hệ thống còn hoạt động
Biểu đồ môi trường	Nhiệt độ và độ ẩm theo thời gian	Theo dõi quá nhiệt và điều kiện môi trường
Biểu đồ rung động miền thời gian	Gia tốc X, Y, Z hoặc RMS	Quan sát dao động tức thời
Biểu đồ FFT	Biên độ theo tần số 0-250 Hz	Nhận diện dải tần bất thường
Log dữ liệu	Chuỗi JSON, trạng thái parse, lỗi kết nối	Hỗ trợ debug khi bảo vệ
Cảnh báo	Normal/Warning/Emergency	Đưa ra phản hồi nhanh cho người vận hành

Giao diện được bố trí theo hướng dễ quan sát trong quá trình vận hành. Khu vực trạng thái kết nối giúp xác nhận ESP32 còn truyền dữ liệu ổn định. Biểu đồ nhiệt độ và độ ẩm cho phép theo dõi điều kiện môi trường xung quanh thiết bị. Biểu đồ rung động miền thời gian thể hiện sự thay đổi tức thời của tín hiệu gia tốc. Biểu đồ FFT hỗ trợ đánh giá các thành phần tần số nổi bật, từ đó phục vụ việc nhận biết các dấu hiệu bất thường như rung mạnh, mất cân bằng hoặc va đập cơ khí.

Khu vực cảnh báo được thiết kế để phản hồi nhanh khi dữ liệu vượt ngưỡng cho phép. Trạng thái hoạt động có thể được chia thành các mức như Normal, Warning hoặc Emergency. Khi phát hiện trạng thái bất thường, phần mềm có thể đổi màu cảnh báo, ghi log sự kiện và hiển thị thông tin để người vận hành kiểm tra thiết bị kịp thời.



Hình 3.13. Giao diện phần mềm giám sát trung tâm khi nhận dữ liệu từ ESP32

Thiết kế phần mềm giám sát theo cấu trúc trên giúp người dùng theo dõi đồng thời nhiều nhóm thông tin: môi trường, rung động, phổ FFT, trạng thái kết nối và cảnh báo. Đây là phần liên kết trực tiếp giữa nút cảm biến và người vận hành, giúp dữ liệu đo được trở nên trực quan hơn, đồng thời tạo nền tảng để bổ sung các chức năng như lưu lịch sử, xuất báo cáo, thống kê lỗi hoặc quản lý nhiều nút cảm biến trong các phiên bản phát triển tiếp theo.

3.8. Ưu điểm và hạn chế của phương án thiết kế

Phương án thiết kế của hệ thống được xây dựng theo hướng tạo ra một nút cảm biến không dây nhỏ gọn, có khả năng thu thập dữ liệu tại vị trí lắp đặt, xử lý một phần dữ liệu ngay trên ESP32 và truyền thông tin về phần mềm giám sát trung tâm. Cách tiếp cận này phù hợp với mục tiêu của đề án là xây dựng một mô hình giám sát tình trạng máy móc có thể hoạt động thực tế, chi phí hợp lý và dễ mở rộng.

3.8.1. Ưu điểm của phương án thiết kế

Một ưu điểm nổi bật của hệ thống là cấu trúc phần cứng gọn và dễ triển khai. Việc sử dụng ESP32 làm bộ xử lý trung tâm giúp giảm số lượng module truyền thông bên ngoài, do vi điều khiển đã tích hợp sẵn Wi-Fi. Các cảm biến MPU6050 và AHT30 có kích thước nhỏ, giao tiếp đơn giản và phù hợp để tích hợp vào một nút cảm biến độc lập.

Thiết bị được thiết kế theo hướng có thể gắn trực tiếp lên bề mặt kim loại của máy bằng nam châm. Nhờ đó, quá trình thử nghiệm thuận tiện hơn, không cần khoan lỗ, bắt vít hoặc thay đổi kết cấu cơ khí của máy cần giám sát. Đặc điểm này phù hợp với các mô hình thử nghiệm trong phòng lab, xưởng nhỏ hoặc các vị trí cần khảo sát nhanh.

Một ưu điểm khác là khả năng xử lý dữ liệu tại biên. ESP32 không chỉ đọc dữ liệu cảm biến mà còn thực hiện các bước xử lý như tính độ lớn rung động, xác định trạng thái cảnh báo và trích xuất thông tin đặc trưng trước khi truyền dữ liệu. Việc này giúp giảm dung lượng gói tin, hạn chế việc truyền toàn bộ dữ liệu thô và hỗ trợ phản hồi nhanh hơn khi có dấu hiệu bất thường.

Phần mềm giám sát trung tâm được xây dựng bằng C# WinForms kết hợp DevExpress, giúp hiển thị dữ liệu dưới dạng trực quan và dễ theo dõi. Người dùng có thể quan sát các thông số như nhiệt độ, độ ẩm, rung động, trạng thái pin và cảnh báo trên cùng một giao diện. Cách triển khai bằng ứng dụng desktop cũng phù hợp với giai đoạn thử nghiệm vì không yêu cầu xây dựng hệ thống máy chủ phức tạp.

Ngoài ra, cấu trúc dữ liệu JSON giúp hệ thống dễ mở rộng. Khi cần bổ sung thêm cảm biến, thêm thông số đo hoặc chuyển đổi giao thức truyền thông, có thể thêm các trường dữ liệu mới mà không cần thay đổi toàn bộ cấu trúc chương trình. Đây là cơ sở để phát triển hệ thống theo hướng nhiều nút cảm biến hoặc tích hợp MQTT trong các phiên bản sau.

3.8.2. Hạn chế của phương án thiết kế

Bên cạnh các ưu điểm, phương án thiết kế hiện tại vẫn còn một số hạn chế. Trước hết, cảm biến MPU6050 là cảm biến MEMS phổ thông, có chi phí thấp và dễ sử dụng nhưng chưa phải là cảm biến rung công nghiệp chuyên dụng. Vì vậy, độ chính xác và độ ổn định của dữ liệu rung động có thể bị ảnh hưởng bởi cách lắp đặt, nhiễu cơ khí và điều kiện môi trường.

Thiết bị sử dụng nguồn pin nên thời gian hoạt động phụ thuộc vào dung lượng pin, chu kỳ lấy mẫu, thời gian truyền Wi-Fi và chế độ tiết kiệm năng lượng. Nếu tần suất gửi dữ liệu cao hoặc thời gian duy trì kết nối Wi-Fi dài, năng lượng tiêu thụ sẽ tăng lên. Do đó, cần tiếp tục tối ưu chế độ Deep Sleep, chu kỳ đánh thức và thời gian truyền dữ liệu để kéo dài thời gian sử dụng.

Về truyền thông, Wi-Fi phù hợp với thử nghiệm trong phạm vi phòng lab hoặc xưởng nhỏ, nhưng có thể bị ảnh hưởng bởi khoảng cách, vật cản kim loại hoặc nhiễu mạng. Khi triển khai ở môi trường công nghiệp thực tế, cần đánh giá thêm độ ổn định của kết nối và có thể cân nhắc các giải pháp như MQTT, lưu đệm dữ liệu khi mất mạng hoặc sử dụng hạ tầng truyền thông phù hợp hơn.

Ngoài ra, hệ thống hiện tại chủ yếu mới dừng ở mức giám sát và minh họa nguyên lý bảo trì dự đoán. Dữ liệu thực nghiệm còn ít, chưa có bộ dữ liệu dài hạn về các trạng thái lỗi thực tế như mất cân bằng, lệch trục, mòn ổ bi hoặc lỏng cơ khí. Vì vậy, hệ thống chưa đủ cơ sở để đưa ra kết luận chẩn đoán lỗi chính xác như các thiết bị chuyên dụng trong công nghiệp.

Phần cơ khí của sản phẩm cũng cần được kiểm thử thêm. Vỏ hộp, nam châm gắn lắp, anten ngoài và dây kết nối cần được đánh giá trong điều kiện rung động kéo dài để đảm bảo thiết bị không bị lỏng, mất kết nối hoặc sai lệch dữ liệu đo. Đây là điểm cần cải thiện nếu muốn phát triển sản phẩm theo hướng sử dụng thực tế lâu dài.

Tổng thể, phương án thiết kế hiện tại phù hợp với mục tiêu xây dựng mô hình thử nghiệm có khả năng thu thập, xử lý và hiển thị dữ liệu giám sát máy móc. Các hạn chế nêu trên không làm giảm ý nghĩa của đề án, mà là cơ sở để xác định những hướng cải tiến cần thực hiện trong các phiên bản phát triển tiếp theo.

CHƯƠNG 4. THI CÔNG SẢN PHẨM VÀ QUY TRÌNH KIỂM THỬ

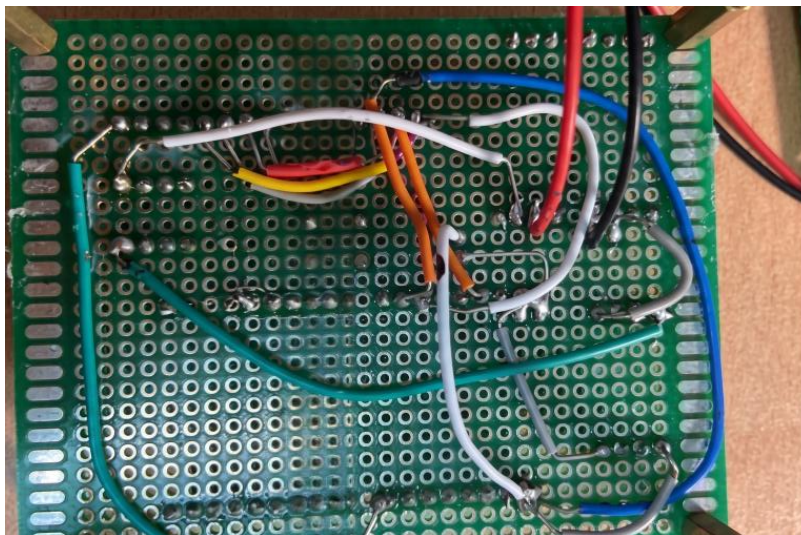
4.1. Quy trình thi công sản phẩm

Quá trình thi công sản phẩm được thực hiện theo từng bước nhằm đảm bảo các khối chức năng của hệ thống hoạt động ổn định trước khi lắp ráp hoàn chỉnh. Sản phẩm được triển khai dưới dạng một nút cảm biến không dây, gồm khối xử lý ESP32, cảm biến rung động MPU6050, cảm biến nhiệt độ - độ ẩm AHT30, khối nguồn dùng pin và phần vỏ hộp bảo vệ.

Trước khi lắp ráp, các linh kiện chính được kiểm tra riêng lẻ để hạn chế lỗi trong quá trình tích hợp. ESP32 được kiểm tra khả năng nạp chương trình và kết nối Wi-Fi. Cảm biến MPU6050 được thử đọc dữ liệu gia tốc theo ba trục, trong khi AHT30 được kiểm tra khả năng trả về giá trị nhiệt độ và độ ẩm. Khối nguồn pin, mạch bảo vệ và mạch ổn áp cũng được đo điện áp đầu ra để đảm bảo phù hợp với mức cấp nguồn cho vi điều khiển và cảm biến.

Sau khi kiểm tra linh kiện, các khối phần cứng được kết nối với ESP32 theo sơ đồ thiết kế. MPU6050 và AHT30 sử dụng giao tiếp I2C nên được nối chung hai đường SDA và SCL. Các đường nguồn, mass và dây tín hiệu được bố trí gọn gàng để giảm rối dây, hạn chế lỏng kết nối khi thiết bị chịu rung động. Đối với khối nguồn, dây pin và công tắc được cố định chắc chắn nhằm tránh chập mạch hoặc đứt dây trong quá trình thử nghiệm.

Tiếp theo, firmware được nạp vào ESP32 để kiểm tra toàn bộ chu trình hoạt động của nút cảm biến. Ở giai đoạn này, dữ liệu cảm biến được quan sát trước qua Serial Monitor để xác nhận cảm biến trả về giá trị hợp lý. Sau đó, ESP32 được cấu hình để đóng gói dữ liệu JSON và truyền về phần mềm giám sát trên máy tính. Việc kiểm tra theo từng bước giúp dễ phát hiện lỗi, chẳng hạn lỗi kết nối cảm biến, sai địa chỉ I2C, lỗi nguồn cấp hoặc lỗi truyền dữ liệu Wi-Fi.



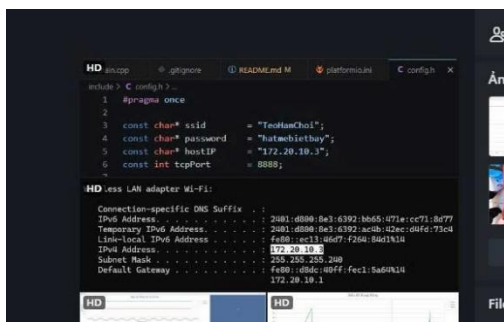
Hình 4.1. Quá trình thi công và kiểm tra mạch của nút cảm biến

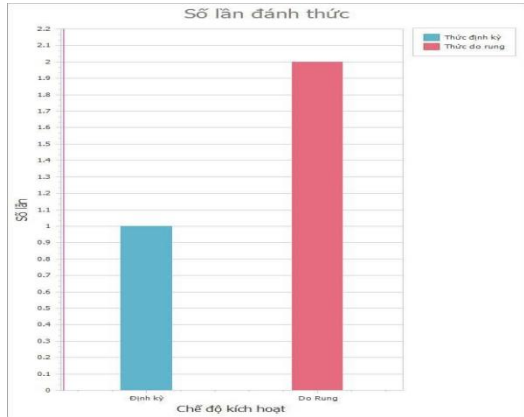
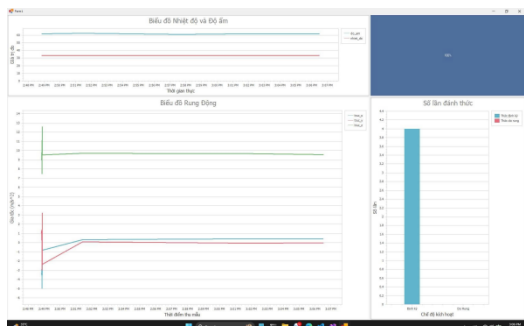
Sau khi các khối điện tử hoạt động ổn định, bo mạch, pin, anten và các dây kết nối được bố trí vào trong vỏ hộp. Vị trí lắp đặt cần đảm bảo cảm biến rung động được cố định chắc chắn để thu nhận dao động tốt, đồng thời anten ngoài được đưa ra ngoài vỏ nhằm cải thiện chất lượng kết nối Wi-Fi. Các chi tiết như công tắc, đầu nối và dây nguồn được sắp xếp gọn để thuận tiện cho thao tác bật/tắt và bảo trì.

4.2. Kế hoạch kiểm thử chức năng

Sau khi hoàn thành quá trình lắp ráp, sản phẩm cần được kiểm thử theo từng nhóm chức năng để đánh giá mức độ hoạt động của hệ thống. Việc kiểm thử được thực hiện theo trình tự từ khối phần cứng cơ bản đến phần mềm giám sát, nhằm dễ dàng phát hiện lỗi và xác định nguyên nhân khi hệ thống chưa hoạt động đúng yêu cầu.

Bảng 4.1. Kế hoạch kiểm thử sản phẩm thực tế

Mã test	Chức năng kiểm thử	Cách tiến hành	Kết quả	Minh chứng
TC01	Nguồn và ổn áp	Đo điện áp sau mạch giảm áp khi cấp pin	Điện áp ổn định, ESP32 khởi động bình thường	
TC02	Kết nối Wi-Fi/TCP	Chạy ESP32 và phần mềm PC cùng mạng	PC nhận được gói JSON liên tục	
TC03	Đọc AHT30	Thay đổi nhiệt độ/độ ẩm môi trường nhẹ	Giá trị nhiệt độ/độ ẩm thay đổi hợp lý	Xem biểu đồ tại hình 5.3
TC04	Đọc MPU6050	Lắc nhẹ sản phẩm theo các hướng	Giá trị X/Y/Z hoặc RMS thay đổi rõ	Xem biểu đồ tại hình 5.4
TC05	FFT rung chậm	Xoay/lắc chậm bằng tay	Phổ tập trung ở vùng tần số thấp	Xem biểu đồ tại hình 5.5
TC06	Giả lập cân bằng	Dán vật nhô lên cánh quạt hoặc quay toàn	Xuất hiện đỉnh phổ rõ tại tần số quay hoặc vùng thấp	Xem biểu đồ tại hình 5.6

Mã test	Chức năng kiểm thử	Cách tiến hành	Kết quả	Minh chứng
TC07	Giả lập ma sát/rung nhanh	Chạm nhẹ vật mềm vào mép quạt hoặc tạo rung nhanh có kiểm soát	Năng lượng phổ tăng ở dải cao hơn	Xem biểu đồ tại hình 5.7
TC08	Deep Sleep/đánh thức	Thiết lập chu kỳ thức định kỳ và thử ngắt rung	Thiết bị thức dậy đúng chu kỳ hoặc khi có rung vượt ngưỡng	
TC09	Cảnh báo phần mềm	Đặt ngưỡng thấp để kích hoạt cảnh báo	Dashboard chuyển trạng thái Warning/Emergency	
TC10	Độ ổn định truyền dữ liệu	Cho hệ thống chạy liên tục 15-30 phút	Không treo chương trình, số gói lỗi thấp	

4.3. Tiêu chí đánh giá sản phẩm

Sau khi hoàn thành quá trình thi công và kiểm thử, sản phẩm được đánh giá dựa trên mức độ đáp ứng các yêu cầu đặt ra ban đầu. Các tiêu chí đánh giá tập trung vào khả năng hoạt động ổn định của phần cứng, độ tin cậy của dữ liệu cảm biến, khả năng truyền dữ liệu không dây và mức độ trực quan của phần mềm giám sát.

Trước hết, sản phẩm cần khởi động ổn định khi cấp nguồn bằng pin. ESP32 phải hoạt động bình thường, không bị reset liên tục trong quá trình đọc cảm biến hoặc truyền

dữ liệu Wi-Fi. Các cảm biến MPU6050 và AHT30 cần trả về dữ liệu hợp lý, có sự thay đổi tương ứng khi thay đổi trạng thái rung động hoặc điều kiện môi trường xung quanh.

Đối với chức năng truyền dữ liệu, hệ thống cần gửi được gói JSON từ ESP32 về phần mềm giám sát trung tâm. Dữ liệu nhận được phải đúng cấu trúc, không bị lỗi định dạng và được cập nhật liên tục trong quá trình thử nghiệm. Phần mềm C# WinForms cần phân tách được các trường dữ liệu, hiển thị đúng thông số nhiệt độ, độ ẩm, rung động, pin và trạng thái cảnh báo.

Về khả năng giám sát rung động, sản phẩm cần thể hiện được sự khác biệt giữa trạng thái đặt yên, rung nhẹ, rung mạnh hoặc giả lập mất cân bằng. Khi thiết bị chịu rung động, giá trị gia tốc, độ lớn rung động tổng hợp hoặc tần số đỉnh phải có sự thay đổi rõ ràng. Đây là cơ sở để chứng minh hệ thống có khả năng phản ánh tình trạng vận hành của máy thông qua dữ liệu cảm biến.

Bên cạnh chức năng đo và truyền dữ liệu, kết cấu cơ khí của sản phẩm cũng cần được đánh giá. Thiết bị phải được cố định chắc chắn trong vỏ hộp, dây nguồn và dây tín hiệu không bị lỏng khi có rung động. Anten ngoài cần được lắp đặt ổn định để duy trì kết nối Wi-Fi. Để nam châm phải giúp thiết bị bám được lên bề mặt kim loại trong quá trình thử nghiệm mà không làm ảnh hưởng đến kết cấu của máy.

Các tiêu chí đánh giá chính gồm:

1. Sản phẩm khởi động ổn định khi cấp nguồn bằng pin
2. ESP32 kết nối Wi-Fi và truyền dữ liệu về máy tính giám sát
3. Cảm biến AHT30 đo được nhiệt độ và độ ẩm với giá trị hợp lý
4. Cảm biến MPU6050 phản ánh được sự thay đổi rung động theo ba trục
5. Gói dữ liệu JSON có cấu trúc đúng và được phần mềm C# xử lý được
6. Giao diện giám sát hiển thị được dữ liệu theo thời gian thực
7. Trạng thái cảnh báo thay đổi phù hợp khi vượt ngưỡng thiết lập
8. Cơ khí sản phẩm chắc chắn, dễ lắp đặt và phù hợp với mô hình thử nghiệm
9. Có đầy đủ hình ảnh minh chứng cho quá trình thi công, lắp đặt và kiểm thử

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Kết quả triển khai sản phẩm

Sau quá trình thiết kế và thi công, sản phẩm được hoàn thiện dưới dạng một nút cảm biến không dây phục vụ giám sát tình trạng máy móc công nghiệp. Thiết bị sử dụng ESP32 làm bộ xử lý trung tâm, kết hợp với cảm biến MPU6050 để thu thập dữ liệu rung động và cảm biến AHT30 để đo nhiệt độ, độ ẩm môi trường. Các dữ liệu sau khi thu thập được xử lý tại nút cảm biến, đóng gói thành dữ liệu JSON và truyền về phần mềm giám sát trung tâm thông qua kết nối Wi-Fi.

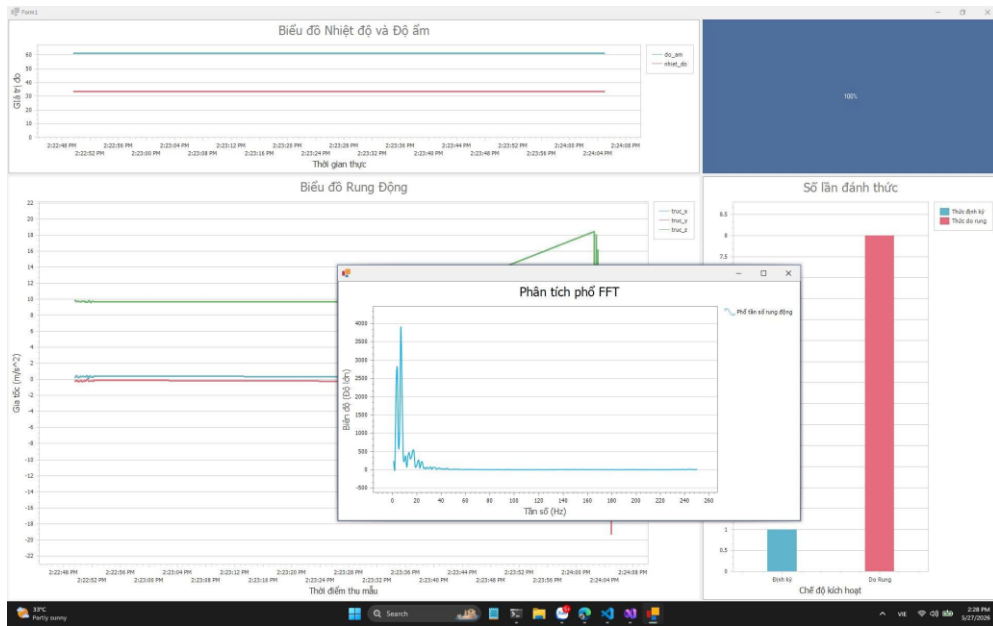
Sản phẩm được bố trí trong vỏ hộp bảo vệ nhằm tăng độ chắc chắn và thuận tiện khi lắp đặt thử nghiệm. Anten ngoài được đưa ra khỏi vỏ hộp để cải thiện khả năng thu phát Wi-Fi. Ngoài ra, thiết bị được thiết kế theo hướng có thể gắn lên bề mặt kim loại của máy bằng nam châm, giúp việc thử nghiệm trên quạt, động cơ hoặc mô hình máy móc trở nên linh hoạt hơn.



Hình 5.1. Sản phẩm sau khi hoàn thiện

Khi cấp nguồn, ESP32 khởi động, kiểm tra các cảm biến và bắt đầu chu trình thu thập dữ liệu. Dữ liệu từ MPU6050 phản ánh dao động theo ba trục, còn dữ liệu từ AHT30 thể hiện điều kiện nhiệt độ và độ ẩm xung quanh thiết bị. Các thông tin này được gửi về máy tính giám sát để hiển thị và theo dõi theo thời gian thực.

Trong quá trình thử nghiệm ban đầu, hệ thống cho thấy các khối chức năng chính có thể phối hợp với nhau: cảm biến trả về dữ liệu, ESP32 xử lý và tạo gói JSON, kết nối Wi-Fi được thiết lập và phần mềm giám sát nhận được dữ liệu từ nút cảm biến. Đây là cơ sở để tiếp tục thực hiện các bước kiểm thử chi tiết hơn ở các mục sau, bao gồm kiểm tra dữ liệu môi trường, dữ liệu rung động, trạng thái cảnh báo và kết quả phân tích tần số.



Hình 5.2. Sản phẩm hoạt động và gửi dữ liệu trong quá trình thử nghiệm

Kết quả triển khai cho thấy hệ thống đã hình thành đầy đủ các khối cần thiết của một mô hình giám sát IIoT cơ bản, gồm khối cảm biến, khối xử lý biên, khối truyền thông và phần mềm giám sát. Mặc dù sản phẩm vẫn cần được kiểm thử thêm trong thời gian dài để đánh giá độ ổn định, phiên bản hiện tại đã đáp ứng được yêu cầu trình diễn nguyên lý thu thập dữ liệu, truyền dữ liệu không dây và giám sát tình trạng máy móc ở quy mô thực nghiệm.

5.2. Kết quả kiểm thử dữ liệu cảm biến

Sau khi sản phẩm được cấp nguồn và kết nối với phần mềm giám sát, các cảm biến được kiểm thử để đánh giá khả năng thu thập dữ liệu thực tế. Nội dung kiểm thử tập trung vào hai nhóm dữ liệu chính: dữ liệu môi trường từ cảm biến AHT30 và dữ liệu rung động từ cảm biến MPU6050.

Đối với cảm biến AHT30, hệ thống ghi nhận hai thông số là nhiệt độ và độ ẩm môi trường xung quanh thiết bị. Trong điều kiện phòng thí nghiệm, giá trị nhiệt độ và độ ẩm cần thay đổi chậm, không dao động đột ngột và nằm trong khoảng hợp lý so với điều kiện thực tế. Khi thay đổi nhẹ môi trường xung quanh cảm biến, chẳng hạn đưa tay lại gần hoặc thay đổi vị trí đặt thiết bị, giá trị đo cần có sự biến thiên tương ứng.

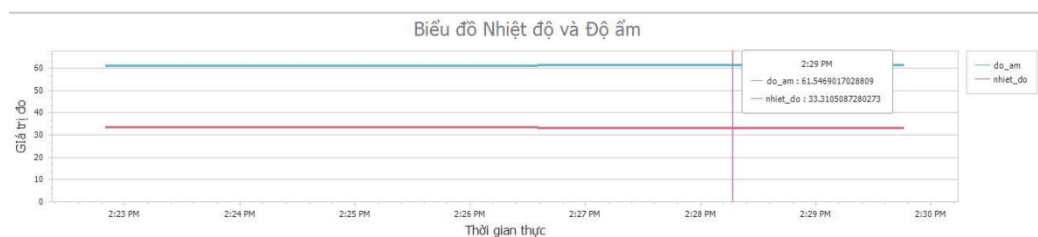
Đối với cảm biến MPU6050, quá trình kiểm thử được thực hiện ở hai trạng thái: đặt yên thiết bị và tạo rung động bằng cách lắc nhẹ hoặc gắn lên mô hình thử nghiệm. Khi thiết bị đặt yên, giá trị gia tốc theo các trục X, Y, Z tương đối ổn định, độ lớn rung động tổng hợp ở mức thấp. Khi thiết bị chịu rung, các giá trị gia tốc và độ lớn rung động tổng hợp tăng lên rõ rệt. Điều này cho thấy cảm biến có khả năng phản ánh sự thay đổi dao động cơ khí của thiết bị.

Kết quả kiểm thử dữ liệu cảm biến được ghi nhận trong Bảng 5.1.

Bảng 5.1. Bảng ghi nhận kết quả kiểm thử cảm biến

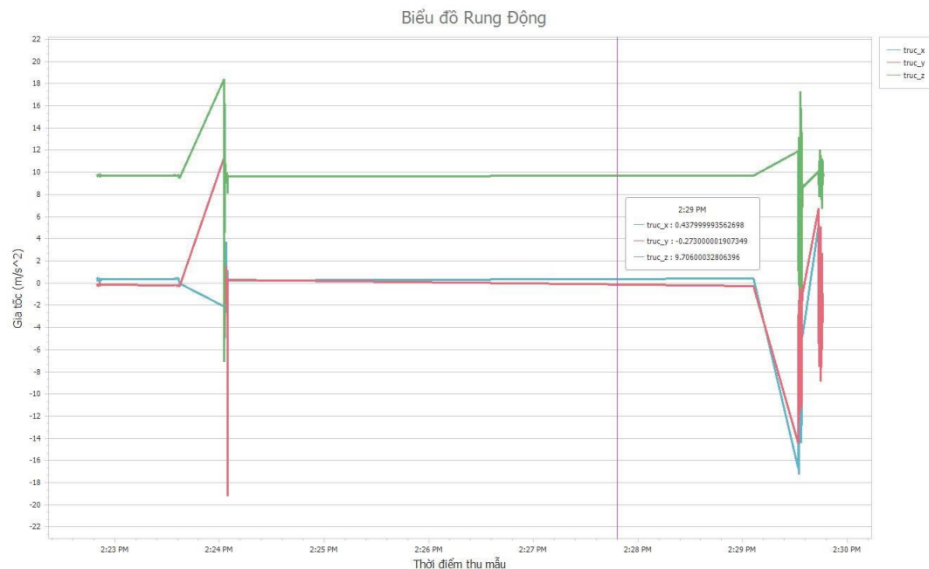
Thông số	Điều kiện kiểm thử	Giá trị/biểu hiện mong đợi	Kết quả thực tế	Đánh giá
Điện áp cấp cho ESP32 và cảm biến	Đo đầu ra khỏi ổn áp bằng đồng hồ vạn năng	Điện áp xấp xỉ 3.3 VDC, đủ cấp cho ESP32 và cảm biến	3.32V	Đạt
Nhiệt độ AHT30	Môi trường trong phòng	Giá trị ổn định và biến thiên chậm	Xấp xỉ 33.3°C	Đạt
Độ ẩm AHT30	Môi trường trong phòng	Giá trị hợp lý theo điều kiện phòng	Xấp xỉ 61.5%RH	Đạt
Gia tốc trục X từ MPU6050	Thiết bị đang truyền dữ liệu lên dashboard	Giá trị thay đổi theo trạng thái rung động của thiết bị	Xấp xỉ 0.44 m/s ²	Đạt
Gia tốc trục Y từ MPU6050	Thiết bị đang truyền dữ liệu lên dashboard	Giá trị thay đổi theo trạng thái rung động của thiết bị	Xấp xỉ -0.27 m/s ²	Đạt
Gia tốc trục Z từ MPU6050	Thiết bị đang truyền dữ liệu lên dashboard	Khi thiết bị gần trạng thái tĩnh, trục Z có giá trị gần gia tốc trọng trường	Xấp xỉ 9.71 m/s ²	Đạt
Gia tốc tổng hợp	Tính từ ba trục X, Y, Z	Giá trị phản ánh mức rung tổng hợp của thiết bị	Xấp xỉ 9.72 m/s ²	Đạt

Sau khi thu thập dữ liệu, phần mềm giám sát hiển thị các thông số nhiệt độ và độ ẩm dưới dạng giá trị số hoặc biểu đồ theo thời gian. Biểu đồ này giúp người dùng quan sát xu hướng thay đổi của môi trường xung quanh thiết bị trong quá trình vận hành.



Hình 5.3. Kết quả đo nhiệt độ và độ ẩm từ cảm biến AHT30

Đối với dữ liệu rung động, phần mềm hiển thị giá trị gia tốc theo các trục hoặc độ lớn rung động tổng hợp. Khi có tác động rung, biểu đồ cần thể hiện sự thay đổi rõ so với trạng thái đặt yên. Kết quả này là cơ sở ban đầu để tiếp tục phân tích phổ FFT và đánh giá các trạng thái bất thường ở mục sau.



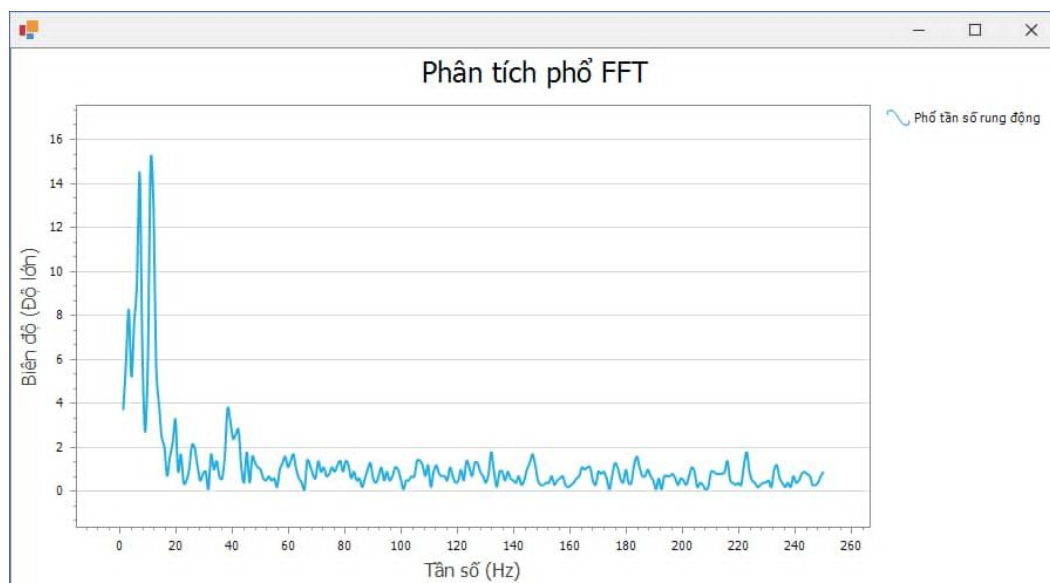
Hình 5.4. Kết quả đo rung động từ cảm biến MPU6050

Qua quá trình kiểm thử, dữ liệu từ AHT30 và MPU6050 có thể được sử dụng để đánh giá trạng thái môi trường và dao động cơ khí của thiết bị. Đây là hai nhóm dữ liệu đầu vào quan trọng cho hệ thống giám sát bảo trì máy móc, đồng thời là cơ sở để thực hiện các bước phân tích cảnh báo và nhận diện bất thường trong các thử nghiệm tiếp theo.

5.3. Kết quả phân tích FFT và giả lập lỗi

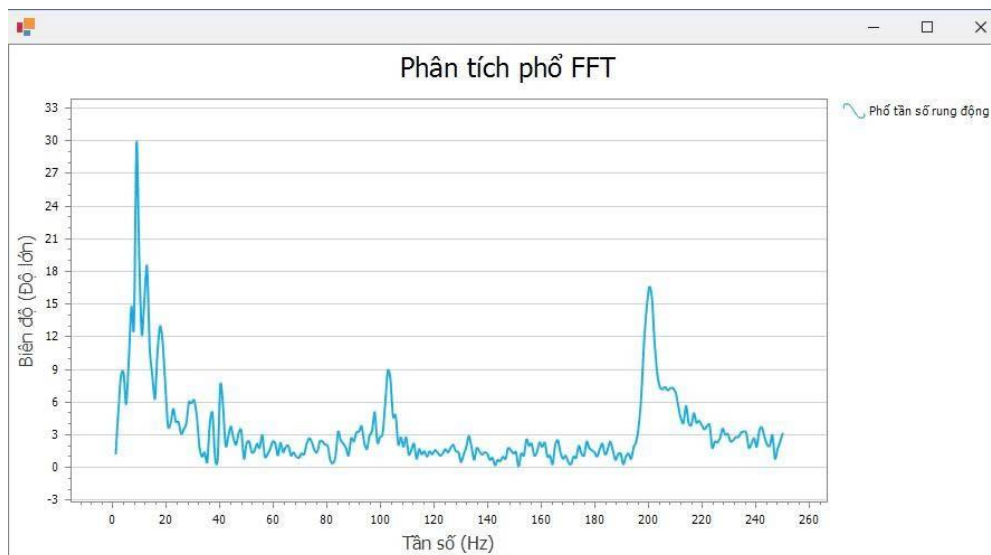
Sau khi kiểm tra dữ liệu cảm biến, hệ thống tiếp tục được thử nghiệm với chức năng phân tích rung động bằng FFT. Mục đích của bước này là quan sát sự thay đổi của tín hiệu rung trong miền tần số khi thiết bị ở trạng thái bình thường và khi tạo ra các tình huống rung động bất thường có kiểm soát.

Trong quá trình thử nghiệm, sản phẩm được gắn lên mô hình quạt hoặc động cơ nhỏ. Khi thiết bị hoạt động ổn định, tín hiệu rung động thường có biên độ thấp và phổ tần số không xuất hiện nhiều đỉnh lớn bất thường. Trạng thái này được xem là mốc tham chiếu ban đầu để so sánh với các trường hợp giả lập lỗi.



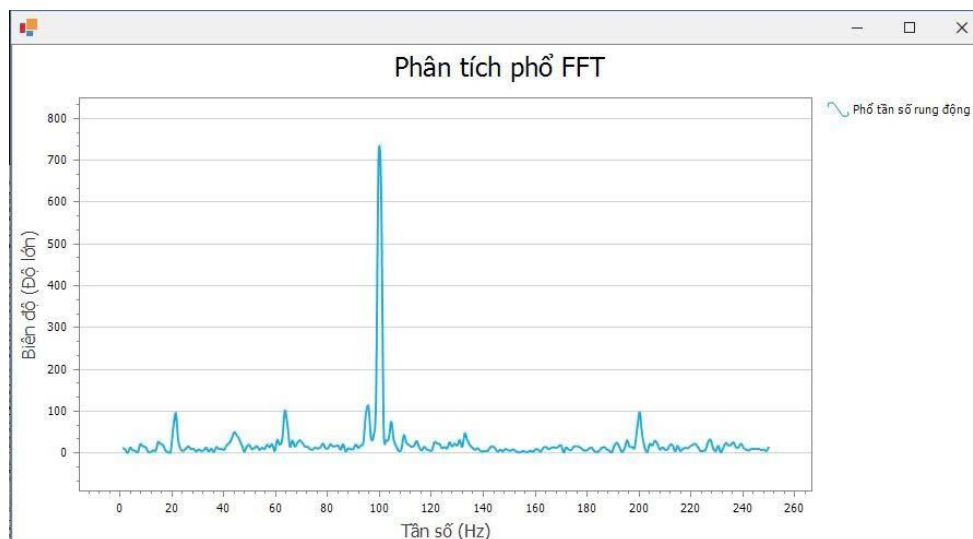
Hình 5.5. Phổ FFT ở trạng thái hoạt động bình thường

Để kiểm tra khả năng nhận biết sự thay đổi rung động, có thể tạo tình huống mất cân bằng bằng cách gắn thêm một vật nhỏ lên cánh quạt hoặc trục quay trong điều kiện an toàn. Khi khối lượng phân bố không đều, dao động của mô hình thử nghiệm tăng lên, làm cho biên độ rung động và tần số đỉnh có sự thay đổi so với trạng thái bình thường. Trên biểu đồ FFT, sự thay đổi này có thể thể hiện qua các đỉnh phổ nổi bật hơn tại một số vùng tần số.



Hình 5.6. Kết quả FFT khi giả lập mất cân bằng cơ khí

Ngoài trường hợp mất cân bằng, hệ thống cũng có thể được kiểm tra bằng cách tạo rung nhanh hoặc ma sát nhẹ có kiểm soát. Khi có tác động rung nhanh, năng lượng của tín hiệu có xu hướng xuất hiện ở vùng tần số cao hơn so với rung chậm. Kết quả này giúp minh họa khả năng của hệ thống trong việc phản ánh sự khác nhau giữa các dạng rung động.



Hình 5.7. Kết quả FFT khi giả lập rung nhanh hoặc ma sát

Thông qua các thử nghiệm trên, có thể đánh giá rằng dữ liệu FFT giúp biểu diễn đặc trưng rung động rõ hơn so với việc chỉ quan sát giá trị gia tốc tức thời. Biểu đồ phổ tần số cho phép người vận hành nhận biết sự thay đổi của dao động khi trạng thái máy thay đổi, từ đó hỗ trợ quá trình giám sát và phát hiện bất thường.

Do mô hình thử nghiệm trong đồ án còn ở quy mô nhỏ, kết quả FFT chủ yếu dùng để minh họa nguyên lý phân tích rung động và so sánh giữa các trạng thái vận hành. Để ứng dụng cho chẩn đoán lỗi công nghiệp chính xác hơn, hệ thống cần được kiểm thử thêm trên nhiều loại máy, nhiều mức tải và trong thời gian dài để xây dựng cơ sở dữ liệu tham chiếu đáng tin cậy.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã thực hiện thiết kế và triển khai hệ thống giám sát, hỗ trợ bảo trì máy công nghiệp theo hướng ứng dụng IIoT. Sản phẩm được xây dựng dưới dạng một nút cảm biến không dây sử dụng ESP32 làm bộ xử lý trung tâm, kết hợp với cảm biến MPU6050 để thu thập dữ liệu rung động và cảm biến AHT30 để đo nhiệt độ, độ ẩm môi trường. Dữ liệu sau khi thu thập được xử lý tại thiết bị biên, đóng gói theo định dạng JSON và truyền về phần mềm giám sát trung tâm.

Về phần cứng, sản phẩm đã hình thành đầy đủ các khối chức năng cơ bản gồm khối xử lý, khối cảm biến, khối nguồn, khối truyền thông và phần vỏ hộp bảo vệ. Thiết bị được thiết kế theo hướng nhỏ gọn, có khả năng gắn lên bề mặt kim loại của máy móc bằng nam châm, thuận tiện cho quá trình thử nghiệm và di chuyển giữa nhiều vị trí đo khác nhau.

Về phần mềm nhúng, ESP32 đảm nhiệm các nhiệm vụ đọc dữ liệu cảm biến, tính toán đặc trưng rung động, xác định trạng thái cảnh báo, quản lý chế độ đánh thức và truyền dữ liệu về máy tính giám sát. Cấu trúc firmware được tổ chức theo chu trình xử lý rõ ràng, giúp hệ thống có khả năng thu thập dữ liệu và phản hồi trạng thái trong quá trình vận hành.

Về phần mềm giám sát, giao diện C# WinForms kết hợp DevExpress giúp hiển thị các thông số đo được theo thời gian thực. Người dùng có thể theo dõi dữ liệu nhiệt độ, độ ẩm, rung động, trạng thái pin, trạng thái cảnh báo và các thông tin liên quan đến quá trình hoạt động của nút cảm biến. Việc trực quan hóa dữ liệu giúp quá trình quan sát và đánh giá tình trạng máy trở nên rõ ràng hơn so với việc chỉ theo dõi dữ liệu dạng số.

Qua quá trình thực hiện, đồ án đã đáp ứng được mục tiêu xây dựng một mô hình giám sát tình trạng máy ở quy mô thử nghiệm. Sản phẩm thể hiện được nguyên lý cơ bản của hệ thống bảo trì dự đoán: thu thập dữ liệu từ thiết bị, xử lý tại biên, truyền dữ liệu không dây và hiển thị trên phần mềm giám sát. Tuy nhiên, do giới hạn về thời gian và điều kiện thử nghiệm, hệ thống hiện tại chủ yếu dừng ở mức mô hình minh chứng nguyên lý, chưa đủ dữ liệu dài hạn để đánh giá chính xác các dạng hư hỏng công nghiệp phức tạp.

6.2. Hướng phát triển

Để nâng cao tính ứng dụng và độ tin cậy của hệ thống, sản phẩm có thể được phát triển thêm theo các hướng sau:

Hoàn thiện thiết kế phần cứng: Thiết kế PCB riêng thay cho mạch thử nghiệm nhằm giảm rối dây, tăng độ chắc chắn và cải thiện khả năng chống nhiễu. Bên cạnh đó, cần tối ưu vị trí đặt cảm biến rung động để tín hiệu thu được phản ánh chính xác hơn dao động của máy.

Bổ sung cảm biến đo điện năng: Hệ thống có thể tích hợp thêm cảm biến dòng điện, điện áp hoặc công suất để kết hợp dữ liệu điện với dữ liệu rung động và nhiệt độ. Việc kết hợp nhiều nhóm thông số sẽ giúp đánh giá tình trạng máy toàn diện hơn.

Cải thiện vỏ hộp và khả năng lắp đặt: Vỏ hộp có thể được thiết kế lại theo hướng chắc chắn, gọn hơn và có khả năng chống bụi, chống ẩm tốt hơn. Cơ cấu gá nam châm

cũng cần được kiểm tra thêm để đảm bảo thiết bị bám chắc khi lắp trên các bề mặt máy có rung động lớn.

Tối ưu quản lý năng lượng: Do thiết bị sử dụng nguồn pin, cần tiếp tục tối ưu chế độ Deep Sleep, chu kỳ đánh thức, thời gian truyền dữ liệu và mức tiêu thụ của các cảm biến. Điều này giúp kéo dài thời gian vận hành của nút cảm biến trong điều kiện thực tế.

Mở rộng giao thức truyền thông; Phiên bản hiện tại có thể sử dụng TCP Socket để truyền dữ liệu trực tiếp về máy tính. Trong các phiên bản sau, hệ thống có thể mở rộng sang MQTT để phù hợp hơn với mô hình IIoT nhiều nút cảm biến, dễ quản lý topic và thuận tiện khi kết nối với nền tảng giám sát khác.

Lưu trữ và phân tích dữ liệu dài hạn: Phần mềm giám sát có thể bổ sung cơ sở dữ liệu để lưu lịch sử nhiệt độ, độ ẩm, rung động và trạng thái cảnh báo. Dữ liệu dài hạn giúp người vận hành theo dõi xu hướng thay đổi của máy, từ đó đánh giá quá trình xuống cấp của thiết bị.

Ứng dụng học máy cho bảo trì dự đoán: Khi có đủ dữ liệu thực nghiệm, có thể xây dựng mô hình học máy để phân loại trạng thái máy như bình thường, mất cân bằng, lệch trục, lỏng cơ khí hoặc rung bất thường. Đây là hướng phát triển quan trọng để hệ thống tiến gần hơn đến bài toán bảo trì dự đoán thực tế.

Triển khai nhiều nút cảm biến: Hệ thống có thể mở rộng từ một nút cảm biến lên nhiều nút đặt tại các vị trí khác nhau trên cùng một máy hoặc nhiều thiết bị trong xưởng. Khi đó, phần mềm trung tâm cần bổ sung chức năng quản lý mã thiết bị, trạng thái từng node và so sánh dữ liệu giữa các vị trí đo.

TÀI LIỆU THAM KHẢO

- [1] Adafruit, “Adafruit_MPU6050: Arduino library for MPU6050”, GitHub repository, 2026. https://github.com/adafruit/Adafruit_MPU6050. Truy cập ngày: 3/3/2026.
- [2] Aosong Electronics, “Datasheet AHT30 Temperature and Humidity Sensor”, Aosong AHT30 Datasheet, 2023. https://eleparts.co.kr/data/goods_attach/202306/good-pdf-12751003-1.pdf. Truy cập ngày: 16/03/2026.
- [3] Arduino Forum, “Using ArduinoFFT with an accelerometer to detect vibration frequency”, Arduino Forum discussion, 2019. <https://forum.arduino.cc/t/using-arduinofft-with-an-accelerometer-to-detect-vibration-freq/609323/5>. Truy cập ngày: 5/3/2026.
- [4] Arduino Libraries, “arduinoFFT”, Arduino Libraries, 2026. <https://www.arduino-libraries.info/libraries/arduino-fft>. Truy cập ngày: 19/2/2026.
- [5] B. Blanchon, “ArduinoJson: Efficient JSON serialization for embedded C++”, ArduinoJson Documentation, 2026. <https://arduinojson.org/>. Truy cập ngày: 13/3/2026.
- [6] DevExpress, “Chart Control | WinForms Controls”, DevExpress Documentation, 2023. <https://docs.devexpress.com/WindowsForms/8117/controls-and-libraries/chart-control>. Truy cập ngày: 22/03/2026.
- [7] E. Condes, “arduinoFFT: Fast Fourier Transform for Arduino”, GitHub repository, 2026. <https://github.com/kosme/arduinoFFT>. Truy cập ngày: 22/3/2026.
- [8] Espressif Systems, “ESP32 Series Datasheet, Version 5.2”, Espressif Documentation, 2025. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Truy cập ngày: 23/3/2026.
- [9] Espressif Systems, “ESP32-WROOM-32 Datasheet”, Espressif Documentation, 2025. https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf. Truy cập ngày: 13/2/2026.
- [10] Espressif Systems, “FreeRTOS (IDF) - ESP-IDF Programming Guide”, ESP-IDF Programming Guide, 2026. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html. Truy cập ngày: 13/2/2026.
- [11] M. Romanssini, P. C. C. de Aguirre, L. Compassi-Severo and A. G. Girardi, “A Review on Vibration Monitoring Techniques for Predictive Maintenance of Rotating Machinery”, Eng, MDPI, vol. 4, no. 3, 2023. <https://www.mdpi.com/2673-4117/4/3/102>. Truy cập ngày: 27/2/2026.
- [12] Microsoft, “Windows Forms for .NET documentation”, Microsoft Learn, 2026. <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>. Truy cập ngày: 27/2/2026.
- [13] P. Kolok et al., “Low-Cost IoT-Based Predictive Maintenance Using Vibration”, PMC / Sensors, 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12609400/>. Truy cập ngày: 18/02/2026.
- [14] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification, Revision 3.4”, Product Specification, 2013. <https://cdn.sparkfun.com/datasheets/Components/General%20IC/PS-MPU-6000A.pdf>. Truy cập ngày: 11/3/2026.
- [15] TDK InvenSense, “MPU-6000 and MPU-6050 Register Map and Descriptions, Revision 4.2”, Register Map and Descriptions, 2011. <https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf>. Truy cập ngày: 25/2/2026.
- [16] TRACTIAN, “Condition Monitoring in Predictive Maintenance: See how TRACTIAN IoT sensors work”, YouTube, 2024. <https://youtu.be/26cnTqheTyA?si=ZzwycgoyEdxXEK58>. Truy cập ngày: 2/02/2026.

PHỤ LỤC

1. Link BOM chi tiết:
https://docs.google.com/spreadsheets/d/1JoigboQBVnsIUBzdPxY6d2fK329IzNQ1HaZJwiEFILo/edit?usp=drive_link
2. Link Github: <https://github.com/phandienxauxa/Machine-Maintenance-Monitoring-System->
3. Bảng kế hoạch thực hiện:
<https://docs.google.com/spreadsheets/d/1dqHzu8Eb5WjBPvD9Nua-COuAt-XcYqp24hudepr0yAA/edit?gid=1321338947#gid=1321338947>